

The Abacus Medal

Human Intuition and Mathematical Exploration in Information Sciences

2026

How to Read This Book

The IMU Abacus Medal recognizes outstanding contributions to the mathematical aspects of information sciences. This book is not a catalogue of medals, affiliations, or publication lists. It is a set of invitations to think: what question was hiding inside a technical obstacle, what representation made the obstacle visible, and how can a student learn to search for the next representation?

The official IMU list names Shayan Oveis Gharan as the 2026 recipient and Mark Braverman as the 2022 recipient. The medal continues the Rolf Nevanlinna Prize, which was awarded from 1982 through 2018. This book therefore covers the complete lineage of winners from 1982 onward. There was no 1992 award: the prize is presented every four years, so the sequence moves from 1990 to 1994. Each chapter moves between a human research problem and the mathematics that made progress possible. Historical statements and award descriptions are tied to the sources in the bibliography; formulas are developed as working tools rather than decorative notation.

No specialist background is assumed. Every chapter should first say what its objects are, what question is being asked about them, and why the question is difficult. Technical words are introduced only after the underlying idea has appeared in an ordinary example; notation then serves as a compact record of an idea the reader has already met. The exercises are intended to reinforce that progression rather than test unexplained vocabulary.

Every chapter follows the same eight-part arc, so the book can be read one chapter at a time or dipped into in any order. A brief note in *italic* opens the chapter naming the single habit of mind its subject rewards. Then: an opening story that meets the ideas in an ordinary situation; the problem, stated in everyday terms before any formalism; the world before, showing what was known and what blocked the field; the new idea, the representation or reframing that made the obstacle visible; worked examples, where the machinery is actually run on small inputs; the mathematics in brief, a compact record of definitions and claims with the reason each is true; and limits and modern views, connecting the result to what came after. Each chapter closes with a second *italic* note, an exercise in judgment that asks the reader to reuse the chapter's habit on a new situation.

The exercises and self-study guide at the end of every chapter ask you to verify the worked examples yourself, to vary them, and to notice where an argument relies on an assumption. Each exercise is followed by a small-print hint, so the book works in both directions: hide the hints to test yourself fully, or keep them visible to keep moving. The most useful way to read a chapter is actively. Before a subsection explains a theorem, pause and ask: what is the input, what would count as an answer, and what information would a direct method have to remember? Only then ask what representation makes the work smaller. Use the exercises to turn admiration into practice.

Contents

1	1982:Robert Endre Tarjan	1
2	1986:Leslie Valiant	8
3	1990:Alexander A. Razborov	18
4	1994:Avi Wigderson	27
5	1998:Peter W. Shor	36
6	2002:Madhu Sudan	46
7	2006:Jon Kleinberg	56
8	2010:Daniel Spielman	67
9	2014:Subhash Khot	77
10	2018:Constantinos Daskalakis	86
11	2022:Mark Braverman	95
12	2026:Shayan Oveis Gharan	102
	Sources and Further Reading	109

Chapter 1

1982:Robert Endre Tarjan

The central habit in this chapter is to let a data structure remember the past, and to judge an algorithm over a whole sequence of operations rather than one isolated step.

1.1 An Opening Story: Joining Clubs and Checking Roads

Six friends—Alice, Bob, Carol, Dave, Eve, and Frank—start in six separate clubs, each with one member. They begin merging: Alice and Bob form one club, Carol and Dave form another, then Bob and Carol merge their two clubs. Now the question is asked: “Are Alice and Dave in the same club?”

If the computer stores each club as a list of members, answering takes two list scans and an easy check. But suppose the clubs keep merging until thousands of people are involved. Every merge can force a list to be walked end to end, and every query may scan a large list. A program built this way can become unusable long before the problem itself is complicated. The simple question—who belongs to whom—hides a design decision: *what information should the program remember between questions?*

The same issue appears for roads. A map of towns and roads is a *graph*: points (*vertices*) joined by lines (*edges*). A *bridge* is a road whose removal would split the map into two disconnected halves. How can we find every bridge without testing each road by hand?

Tarjan’s 1982 Rolf Nevanlinna Prize recognized two answers that now appear in nearly every computer science curriculum: the *union-find* data structure with amortized analysis, and *depth-first search* with its discovery and low-link numbers. Both answers share one philosophy: the structure should record a small, provably correct summary of the past, and the running time should be measured over a whole sequence of operations.

1.2 The Problem

Two problems, both phrased as questions a computer must answer fast.

Union–find. We maintain a collection of disjoint sets of items. Two operations are allowed:

- **FIND**(x): return the name of the set containing x ;

- **UNION**(x, y): merge the sets containing x and y .

A sequence of m operations on n items arrives, one after another. How much total time is needed? The naive list approach is easy but can be catastrophically slow on long sequences.

Graph exploration. Given a graph, find all bridges, articulation points (vertices whose removal disconnects the graph), and, for directed graphs, strongly connected components (maximal groups in which every vertex can reach every other). A correct but naive method—remove each edge and re-search—repeats a global computation m times.

The problems look unrelated: one is about organizing data, the other about exploring a graph. The connection is that both were stuck because nobody had the right way to *measure* cost and the right thing to *remember*.

1.3 The World Before

In the early decades of computer science, memory was expensive, processors were slow, and an algorithm was typically judged by its worst-case performance on a single operation. Two consequences followed.

Union–find as linked lists. The natural implementation stores each set as a linked list. A **FIND** walks the list of the item’s set; a **UNION** walks one list to its tail and attaches the other list. This is easy to describe and correct. But consider merging n singletons into one set by always appending the whole current set to the next item: the growing list is walked once per merge, costing about $n^2/2$ steps. Each **UNION** is individually cheap—except when it is the n -th one—and every earlier merge looks fine in isolation. There was no language for saying: “this particular operation is expensive, but that is fine because it can only happen rarely.”

Finding bridges by deletion. To test whether a road is a bridge, remove it and check whether the two endpoints are still connected. Repeating this for every edge of a graph with m edges costs about m full connectivity searches. On a large road map, that is hopeless. The naive algorithm works on paper but reveals nothing about the structure that makes some roads fragile and others not.

The missing ingredients were two ideas that seem obvious in hindsight:

1. *Amortized analysis*: judge a sequence of operations, and let rare expensive operations be paid for by many cheap ones.
2. *Reusable certificates*: a graph exploration should record exactly the information a later decision needs, not re-derive it from scratch.

1.4 The New Idea

1.4.1 Union–find: sets as shallow trees

Store each set as a rooted tree, with every item pointing to its parent and a root pointing to itself. $\text{FIND}(x)$ climbs from x to its root and returns it. $\text{UNION}(x,y)$ finds both roots and attaches one under the other.

Two refinements make the trees stay shallow:

- **Union by rank.** Each root carries a *rank*, roughly the height of its tree. Always attach the smaller-rank root under the larger-rank one. This alone guarantees that every tree has height at most $\log_2 n$, so a single FIND never costs more than $O(\log n)$.
- **Path compression.** During $\text{FIND}(x)$, every vertex on the path from x to the root is re-pointed directly at the root. The mathematical partition is unchanged—the representative is the same—but future queries along that path become one step each. The query *rewrites the structure to make later work cheaper*.

The second refinement is the surprising one. A query that “only asks a question” actively improves the data structure. That is only safe if the improvement preserves the invariant that makes answers correct. It does: pointing a vertex at its root does not change which set any item belongs to.

1.4.2 Amortized analysis: the bank account

To understand why the combination is fast, forget single operations. Think of a bank account. Cheap operations deposit credit; an expensive operation spends the accumulated credit. An analysis is successful if we can prove the balance never goes negative.

Formally, assign a number Φ (the *potential*) to the current state of the data structure. If operation i costs c_i and changes the potential from Φ_{i-1} to Φ_i , its *amortized* cost is defined as

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

Summing over a whole sequence makes the potential terms telescope, so the total real cost equals the total amortized cost plus a boundary term. If the potential is nonnegative and each $\hat{c}_i$ is small, the whole sequence is cheap. This is the mathematical version of the bank account: an expensive operation is charged for *raising* the potential, and later cheap operations are the ones that spend it.

For union–find, the potential measures how “squashed” the trees are. Path compression is expensive when it flattens a long path, but it buys a large improvement in the forest; union by rank ensures that any one vertex cannot be moved into a better position too many times.

1.4.3 Depth-first search: two numbers as a certificate

Start at any vertex and follow one unexplored road at a time, turning back only when you reach a dead end. Number each vertex (*disc*, for discovery time) in the order it is first reached. While searching, also record, for each vertex v , the number

$\text{low}(v)$ = the smallest discovery time reachable from the subtree under v

using tree edges followed by at most one non-tree edge. In words: *how far back can this part of the map return, without retracing the parent road?*

Two inequalities now read like the whole answer:

- if a child w of v has $\text{low}(w) > \text{disc}(v)$, then the road v – w is a bridge (the subtree under w cannot return to v or above);
- if $\text{low}(w) \geq \text{disc}(v)$, then removing v separates that child's subtree from the rest (the subtree can return to v itself, but not above it).

One pass over the graph produces all bridges, articulation points, and—with a stack—strongly connected components of a directed graph. The discovery numbers and low-links are the certificate: they summarize an enormous family of possible return paths without listing them.

1.5 How It Works: Worked Examples

1.5.1 A warm-up: amortization with a growing array

Before union–find, see amortization on a friendlier object. A dynamic array must grow when it is full. Doubling the size and copying every item costs $O(n)$ when the array holds n items, which sounds expensive. But over m appends, the array doubles only about $\log_2 m$ times, and the total copying is

$$1 + 2 + 4 + \cdots + 2^{\lceil \log_2 m \rceil} = O(m).$$

The expensive rebuilds are paid for by the many cheap appends that filled the old block. Each append has amortized cost $O(1)$. This is the same accounting that makes path compression work.

1.5.2 Union–find on six people, step by step

Start with A, B, C, D, E, F , each in its own set.

1. UNION(A, B): both roots have rank 0; attach A below B . Tree: $A \rightarrow B$.
2. UNION(C, D): similarly, $C \rightarrow D$.
3. UNION(B, D): roots B and D have equal rank; attach B below D . Now $A \rightarrow B \rightarrow D$ and $C \rightarrow D$. Root: D .

4. $\text{FIND}(A)$: climb $A \rightarrow B \rightarrow D$ and return D . Path compression re-points A and B directly at D . The next $\text{FIND}(A)$ or $\text{FIND}(B)$ costs one step.
5. $\text{UNION}(E, F)$: $E \rightarrow F$.
6. $\text{FIND}(A)$ vs. $\text{FIND}(E)$: returns D and F —different clubs.

Without path compression, step 4 would leave the chain $A \rightarrow B \rightarrow D$ intact forever. With it, the query paid for a permanent shortcut. If a club merge later asks for $\text{FIND}(A)$ a hundred times, the first costs three steps and the remaining ninety-nine cost one each.

1.5.3 A bridge hunt with depth-first search

Take the graph with edges

$$(a, b), (b, c), (c, a), (c, d), (d, e), (e, f), (f, d).$$

Explore from a , always taking the alphabetically first unexplored road.

Vertex	DISC	LOW
a	1	1
b	2	1
c	3	1
d	4	4
e	5	4
f	6	4

How the low-links are computed, from the leaves upward. Vertex f has a back edge to d (discovery time 4), so $\text{low}(f) = 4$. Vertex e sees its child f reaching 4, so $\text{low}(e) = 4$. Vertex d 's subtree reaches 4 and no further, so $\text{low}(d) = 4$. Vertex c has a back edge to a (time 1), so $\text{low}(c) = 1$, and this value propagates up through b .

Now test the bridges:

- edge $c-d$: is $\text{low}(d) = 4 > \text{disc}(c) = 3$? Yes. The subtree $\{d, e, f\}$ cannot return above c —bridge. This matches intuition: the triangle on a, b, c and the cycle on d, e, f are joined only by $c-d$.
- edge $e-f$: $\text{low}(f) = 4 > \text{disc}(e) = 5$? No. Within a cycle, no single road is a bridge.

The search never tests every alternative route. It asks, once per tree edge, whether the child's subtree can climb back over the parent road. The two numbers disc and low contain exactly the information needed and no more.

1.6 The Mathematics in Brief

The famous union–find guarantee is

$$O(m \alpha(n))$$

for m operations on n items, where α is the *inverse Ackermann function*. This function grows so slowly that it is at most 4 for any number of atoms in the universe. The notation is a compact record of a real theorem: the total pointer changes across all unions and finds are almost linear. The proof groups vertices by rank and shows that a vertex can be redirected only through a tiny hierarchy of rapidly expanding rank classes. The intuition is simple even when the hierarchy is not: *every expensive flattening buys a large, durable improvement in the vertex’s future position.*

For depth-first search, the inequalities

$$\text{low}(w) > \text{disc}(v) \implies \text{edge } v\text{--}w \text{ is a bridge}$$

and

$$\text{low}(w) \geq \text{disc}(v) \implies v \text{ is an articulation point for that branch}$$

turn a visual idea about fragile networks into a checkable algorithm. The strict versus non-strict inequality is exactly the difference between “the subtree can return to v but not above it” and “it cannot even reach v again.”

Depth-first search also produces certificates for directed graphs. A stack holds vertices of the search branch currently being explored. When a vertex is identified as the root of a strongly connected component (its low-link equals its discovery time), popping until the root removes exactly one complete component. Each vertex and edge is pushed, inspected, and removed a bounded number of times, so the whole algorithm runs in linear time.

1.7 Limits and Modern Views

The classical guarantees depend on implementation details. A careless union rule creates tall trees and destroys the near-linear bound. If parallel edges are stored without identifiers, a bridge test can mistake one copy of a doubled road for the parent edge. Recursive depth-first search can exhaust the call stack on a very long path; an iterative version must store its own exploration state. The lesson is that the invariant must be stated before the code is written.

Two modern settings push beyond the classical picture:

- **Dynamic graphs.** Real networks change: a road closes, a dependency is deleted. Re-running a linear algorithm after every update is too slow. Deletions are the hard case because an edge that once supported connectivity can vanish. The low-link invariant does not survive a deletion automatically, so dynamic connectivity maintains richer structures—spanning forests, balanced trees, randomized sampling—while asking the same Tarjan-style question: *what summary of the past is enough to repair the answer after a local change?*

- **Parallelism.** Depth-first search is sequential in its purest form, because the next branch depends on which vertices are already marked. Connectivity and union operations parallelize better, but synchronization and memory traffic become part of the real cost. An asymptotically optimal algorithm can lose to a worse one on real hardware if its operations are hard to distribute.

The design philosophy survives all these changes: use a small, explicit certificate of the past to make a large future computation cheap. The certificate may be a parent pointer, a rank, a discovery time, or a low-link value. Its power comes from the proof that it is exactly the information later decisions require [14, 15].

1.8 Exercises and Self-Study Guide

- E1.** Run UNION by rank and FIND with path compression on the people A, B, C, D, E, F with merges (A, C) , (E, D) , (B, F) , (C, D) , (F, A) . Draw the forest after each merge.
Hint: Attach the smaller-rank root under the larger; remember that path compression happens only inside FIND.
- E2.** Show that merging n singletons into one set by the linked-list method can cost about $n^2/2$ steps.
Hint: Keep merging the growing list into the next singleton, and count how many times the growing list is walked.
- E3.** Prove, from the low-link definition, that $\text{low}(w) > \text{disc}(v)$ forces the tree edge $v-w$ to be a bridge.
Hint: If removing $v-w$ still left the graph connected, some path from the subtree under w to the outside would exist; trace how low-link would then reach $\text{disc}(v)$ or less.
- E4.** On the graph $(a, b), (b, c), (c, a), (c, d)$, run depth-first search from a and compute all discovery and low-link numbers. Where is the articulation point?
Hint: Follow the same table format as the worked example. Check which child w of c satisfies $\text{low}(w) \geq \text{disc}(c)$.
- E5.** Design a potential function that proves the amortized $O(1)$ append cost of a doubling array.
Hint: Let the potential be the number of occupied slots beyond half the array's capacity.
- E6.** Kruskal's minimum-spanning-tree algorithm accepts edges from lightest to heaviest, adding an edge exactly when its endpoints lie in different components. Explain which two invariants prove correctness and which data structure provides the speed.
Hint: Separate "the accepted edges form a forest" from "each accepted edge is the lightest across a cut." Union-find maintains only the component information the first invariant needs.

A final exercise in judgment: whenever a query operation improves the structure for the future, check which invariant guarantees the improvement is invisible to the answers. If you cannot name that invariant, the optimization is not safe.

Chapter 2

1986:Leslie Valiant

The central habit in this chapter is to count the witnesses, not merely to decide that one exists, and to judge a learner by a proven promise about unseen examples rather than by its score on the examples it has already seen.

2.1 An Opening Story: Pairing Up for a Quiz Night

A quiz club has four members—Alice, Ben, Chen, and Dana—who must form two teams of two for a friendly round. There is one house rule: two members who have already been teammates this month may not pair up again. Only Alice and Ben have played together so far.

The captain asks the obvious question: “Can we form the two teams at all?” The answer is one bit: yes. Then the captain asks a quieter question: “*How many* different ways are there to split the four members into two allowed teams?” The answer is a number. The full pairings of four people are (Alice,Ben)+(Chen,Dana), (Alice,Chen)+(Ben,Dana), and (Alice,Dana)+(Ben,Chen)—three in total. The first is forbidden, so exactly two survive. The computer had to say 2, not merely “yes,” and it would have had to distinguish 2 from 1 or 3 with equal care. Now grow the club: with 40 members and no restrictions, the number of ways to split into 20 teams is about 3.2×10^{23} , larger than all the stars of the Milky Way, even though “does any split exist?” is still answered by one word. The questions look identical but are different computations: one produces a bit, the other a number that can be astronomically large.

This chapter is the mathematics of answering the second kind of question, and of a parallel discovery about learning: the same discipline that separates “is there one?” from “how many

are there?” also separates “does it fit?” from “will it generalize?”

2.2 The Problem

Counting versus deciding. A *bipartite graph* is a collection of points split into two sides, left L and right R , with lines (*edges*) joining only left to right. A *matching* is a set of edges in which no two edges share an endpoint; a *perfect matching* uses every point exactly once. Think of the left side as people and the right side as jobs, an edge meaning “this person can do this job.” Decision asks whether every person can be assigned a distinct job; counting asks how many complete assignments exist. On a tiny instance— $L = \{a, b, c\}$, $R = \{1, 2, 3\}$, every edge present except $a-1$ —the answers are “yes” and “4.”

The same split appears in logic. A *Boolean formula* is a recipe built from variables that are each true or false. The formula $\varphi = (x \vee y) \wedge (x \vee z)$ over the variables x, y, z is true for exactly 5 of the 8 possible truth choices. “Is there any satisfying choice?” has answer *yes*; “how many are there?” has answer 5.

Why should the second be harder than the first? Because a search can stop the moment it finds one witness, while a count must account for every branch, including those that contain no witness at all. To find a perfect matching you grow an assignment edge by edge, backtracking when a road closes; but backtracking records only “this branch failed,” discarding how many completions it contained. The count is a global property of the whole input; each local yes/no answer reveals almost nothing about it.

Learning a rule from examples. A learner receives examples of the form (features, label). A concrete case: a course records, for each student, the hours studied and whether the student passed. The learner wants a rule—say, a threshold on hours—that predicts the label of the *next* student, who is unseen. Fitting is the easy part: any threshold that agrees with the examples is a candidate. The hard part is a promise about the future: what could justify a statement like “with 76 examples, any threshold that fits them all misclassifies at most 10 in every 100 new cases, with 95% confidence”? That claim concerns data never observed, so fitting cannot verify it; it needs a model in which “generalizes” has a precise meaning and can be proved.

2.3 The World Before

In the 1970s, Cook and Levin proved that “does this Boolean formula have a satisfying assignment?” is the hardest of a whole family of decision problems, and Karp exhibited dozens more with the same status. The central object of the new theory was the *decision problem*: a set of inputs whose answer is yes or no. Complexity was measured by the difficulty of telling yes apart from no, and in the same decade Edmonds showed the matching problem easy for *decision*: polynomial-time algorithms say whether a perfect matching exists and return one if it does.

Because decision was the currency of the field, “solving” a problem came to mean finding a single witness, and the folklore reflex for a count was to run the decision procedure repeatedly. This fails even in miniature. For the formula $\varphi = (x \vee y) \wedge (x \vee z)$, a decision oracle answers “yes,” and each time you ask again it returns one witness, say $(1, 0, 0)$, then another; each answer is a single object. To learn that five exist, you must stop the oracle repeating itself by feeding back what you have already seen—bookkeeping that is enumeration, the counting computation itself. An oracle that answered “are there at least k ?” could binary-search the count, but answering “at least k ?” is itself a counting question.

Learning was in a similar state, but murkier. The folk model was curve fitting: take a few data points and draw a rule through them. A polynomial can pass through any finite set of points and then go anywhere at all; fitting was never the issue, so nobody could say what was being asked. Statisticians had heuristics—model selection, cross-validation, bias–variance trade-offs—but their claims were not theorems with explicit parameters.

Two conceptual tools were missing. First, a complexity theory for *counting functions* rather than languages, with reductions that preserve the number of witnesses rather than merely the answer to a yes/no question. Second, a mathematical model of learning in which the error, the confidence, the number of examples, and the running time all appear as quantities in a single theorem. Valiant supplied both.

2.4 The New Idea

The first idea: *count the witnesses, and treat the count as a function in its own right*. Suppose $R(x, y)$ means “ y is a valid witness for input x ” with two constraints: each witness has length at most polynomial in the size of x , and checking $R(x, y)$ takes polynomial time. Define

$$F_R(x) = |\{y : |y| \leq q(|x|), R(x, y)\}|$$

for some polynomial q . The family of all such functions is $\#P$, read “sharp P”: the functions that count efficiently checkable, polynomially bounded witnesses. Satisfying assignments, perfect matchings, and schedules all fit this form. The conceptual shift is sharp: *before*, complexity theory studied sets and asked whether an input belongs to one; *the new idea* is that “how many witnesses does this input have?” is a different question, whose answer is a possibly enormous integer. Such *parsimonious* reductions—the counting analogue of polynomial-time reductions—must preserve that integer exactly, and they build a hierarchy whose top, $\#SAT$ and the *permanent*, is $\#P$ -complete.

The permanent is the canonical hard counting function:

$$\text{perm}(A) = \sum_{\pi \in S_n} \prod_{i=1}^n A_{i, \pi(i)}.$$

For a zero-one matrix it counts perfect matchings of a bipartite graph. Compare it with the determinant: the same sum, but with a sign $\text{sgn}(\pi)$ in front of every term. The sign lets elimination cancel most of the sum, which is why determinants run in polynomial time; the permanent has no cancellation, and Valiant proved it $\#P$ -complete. A single bit of information—which sign to attach—separates easy from believed-hard.

The second idea, for learning, is an explicit contract rather than a new complexity class. In the *Probably Approximately Correct* (PAC) model, examples are drawn independently from an unknown distribution D . The learner picks a hypothesis h from a class H , and the guarantee is: with probability at least $1 - \delta$ over the sample, the probability that h errs on a fresh draw is at most ϵ . Both the sample size m and the running time are charged. *Before*, people thought learning was fitting; *the new idea* is that learning is a theorem of the form “with m samples, whatever hypothesis the algorithm chooses is good,” where the key move is to prove the guarantee

for *all* hypotheses in H simultaneously—because the chosen one depends on the sample, it cannot be certified after the fact.

2.5 How It Works: Worked Examples

2.5.1 Counting perfect matchings, line by line

Take $L = \{a, b, c\}$, $R = \{1, 2, 3\}$, and every edge except $a-1$. A perfect matching pairs each left point with a distinct right point, so it is exactly a permutation of $\{1, 2, 3\}$ with a not paired with 1. Enumerate:

matching	assignment	uses $a-1$?	allowed?
$a-1, b-2, c-3$	123	yes	no
$a-1, b-3, c-2$	132	yes	no
$a-2, b-1, c-3$	213	no	yes
$a-2, b-3, c-1$	231	no	yes
$a-3, b-1, c-2$	312	no	yes
$a-3, b-2, c-1$	321	no	yes

Of the $6 = 3!$ permutations, the two using the missing edge are discarded, leaving 4. The decision answer is “yes,” one bit; the count must tell this graph, with its 4 perfect matchings, apart from a graph with 1, 2, or 6. A decision algorithm returns a single matching and stops; to reach the number 4 it would have to find all four without repeating one—maintaining the list it has already produced, the counting work in disguise.

2.5.2 The satisfying assignments of a tiny formula

For $\varphi = (x \vee y) \wedge (x \vee z)$, test all 8 assignments:

x	y	z	$(x \vee y)$	$(x \vee y) \wedge (x \vee z)$
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

The count is 5. A more instructive way to see it: split on x . If $x = 1$, both clauses are already satisfied and y, z are free, giving $2 \cdot 2 = 4$ assignments. If $x = 0$, the clauses reduce to $y = 1$ and $z = 1$, giving exactly one. Total: $4 + 1 = 5$. A decision search walks down one branch and stops; only an algorithm that keeps both branches and adds their counts returns 5.

2.5.3 Learning a threshold, with a number attached

Suppose the unknown rule is “pass iff hours ≥ 3 .” Four samples arrive: hours 1 and 2 fail; hours 3.5 and 5 pass. The learner’s class is thresholds $x \geq t$; a natural choice, $t = 2.5$, sits just past the largest negative example and fits the sample, but so does $t = 3.1$. The two hypotheses differ exactly in the strip between 2.5 and 3.1, which the sample never shows. In general, a consistent threshold can err only inside the *gap* between the largest negative and the smallest positive sample point, and its risk is at most the probability mass of that gap.

Now suppose the learner restricts itself to $N = 100$ candidate thresholds, so the class is finite. Fix one candidate whose true error is at least $\varepsilon = 0.1$. The probability that it makes no mistake on m random samples is at most

$$(1 - \varepsilon)^m \leq e^{-\varepsilon m}.$$

By the union bound, the probability that *some* candidate is bad and yet fits the sample is at most $100 e^{-\varepsilon m}$. Forcing this below $\delta = 0.05$ gives

$$m \geq \frac{1}{\varepsilon} \ln \frac{N}{\delta} = 10 \ln 2000 \approx 76.$$

So about 76 samples suffice: with probability at least 95%, every candidate threshold that fits the sample has true error at most 10%, and therefore so does the one the learner chose. The bound is *uniform*—proved for all 100 candidates before the sample is drawn—so it covers whichever hypothesis the data selects; a non-uniform argument would be circular, because the chosen hypothesis is a random function of the very sample used to evaluate it.

2.6 The Mathematics in Brief

A function F belongs to $\#P$ when $F(x) = |\{y : |y| \leq q(|x|), R(x, y)\}|$ for a polynomial q and a relation R checkable in polynomial time: the value counts *witnesses*, and both the witness length and the checking time are polynomially bounded. A *parsimonious reduction* maps instances of one counting problem to instances of another so that the number of witnesses is preserved exactly; $\#SAT$ and the permanent are $\#P$ -complete. Why is the permanent $\#P$ -complete? Valiant reduced $\#SAT$ to the permanent by way of *cycle covers*. The permanent of the adjacency matrix of a directed graph with edge weights $0, \pm 1$ sums, over all ways to cover the vertices with disjoint directed cycles, the product of the weights on those cycles—a sum of exactly the right shape to simulate counting. Each Boolean variable becomes a small gadget with exactly two surviving local configurations, recording its truth value; each clause becomes a gadget whose cycles survive precisely for the assignments that satisfy it. The graph is built so that the only nonzero cycle covers correspond one-for-one with satisfying assignments, making the permanent equal the count. Hence a polynomial-time permanent algorithm would compute $\#SAT$, and hardness has teeth: by Cook–Levin it would put satisfiability (ask whether the count is zero) in polynomial time, and Toda’s theorem extends the reach of a single counting computation beyond the whole polynomial hierarchy. Yet the difficulty is really about magnitude, not about the objects: the permanent and determinant are identical except for signs, and since $-1 \equiv 1 \pmod{2}$, the parity of the number of perfect matchings is computable in polynomial time as a determinant. Counting mod 2 is easy; counting exactly is believed impossible.

The PAC guarantee has the structure

$$\text{true error}(h) \leq \text{training error}(h) + \text{complexity penalty}.$$

For a finite class of size N , the penalty carries a $\ln N$ term; the bound $m \geq \frac{1}{\varepsilon} \ln \frac{N}{\delta}$ follows from the union bound—for a fixed hypothesis with true error at least ε , the chance it looks perfect on m samples is at most $e^{-\varepsilon m}$, multiply by the number of hypotheses, and solve for m . The empirical picture transfers to new data *simultaneously for the whole class*, which makes it valid for a hypothesis chosen by looking at the sample.

2.7 Limits and Modern Views

Exact counting of witnesses is believed impossible in polynomial time, so modern work asks what can be saved. One line, *approximate counting*, accepts a multiplicative estimate and uses Markov chains: design a random walk whose stationary distribution weights each matching correctly, let it mix, and average an observable over the walk. This replaces the impossible exact count with a sampler, but only when the chain mixes fast. The permanent admits a fully polynomial-time randomized approximation scheme for nonnegative matrices (Jerrum, Sinclair, and Vigoda), and the subject is haunted by *bottlenecks*: a chain whose state space is two dense regions joined by a narrow corridor takes exponentially long to cross it. Sampling, counting, and the geometry of the state space are one theorem.

The PAC model has aged in two ways. It is a worst-case guarantee over distributions: it holds for any D , which is honest but demands more than practice needs, so modern learning theory adds structure—VC dimension replaces $\ln N$ for infinite classes, noise and robust estimation change the loss, privacy budgets a new resource. And PAC is only the *statistical* clause of learning. A hypothesis class can be statistically learnable while no efficient algorithm finds the hypothesis, and a rule that generalizes under D can fail when the future differs from the past. Valiant’s insistence that error, confidence, samples, and running time be accounted together remains the frame in which all these refinements are stated [16, 17].

2.8 Exercises and Self-Study Guide

E1. Verify by listing: how many assignments of $(x, y, z) \in \{0, 1\}^3$ satisfy $(x \vee y) \wedge (y \vee z) \wedge (z \vee x)$?

Hint: Only four of the eight assignments are excluded. Group the exclusions by which pair of

variables is both 0.

- E2.** In the worked matching graph, remove a second edge, $b-2$, in addition to $a-1$. Count the perfect matchings using inclusion-exclusion.

Hint: Start from $3! = 6$. The 2 matchings using $a-1$ and the 2 using $b-2$ overlap in the 1 matching using both, so $6 - 2 - 2 + 1 = 3$.

- E3.** Give a counting problem whose decision version is easy but whose count is believed hard, and explain why calling the decision algorithm repeatedly does not help.

Hint: 2-satisfiability is solvable in polynomial time, yet counting its satisfying assignments is #P-complete.

- E4.** Derive the union-bound sample size $m \geq \frac{1}{\epsilon} \ln \frac{N}{\delta}$ from scratch, and re-check the number 76.

Hint: Fix one hypothesis with true error at least ϵ ; bound the chance that it is clean on m samples by $e^{-\epsilon m}$; union over all N and solve for m .

- E5.** Give two thresholds that agree on the sample $\{1, 2, 3.5, 5\}$ but disagree on an unseen region, and state what extra information would distinguish them.

Hint: Any $t \in (2, 3.5]$ is consistent with the sample; two such thresholds differ on the strip between them, where the risk lives.

- E6.** Describe a Markov chain whose states are the perfect matchings of a small bipartite graph, and draw a graph for which the chain has a clear bottleneck.

Hint: Let a step move to a new matching by toggling one or two edges, with proposal probabilities chosen to keep the chain symmetric; a bottleneck is two large families of matchings joined by very few intermediate matchings.

- E7.** Take the claim “a learner trained on these examples will generalize.” Separate it into a statistical clause, a computational clause, and a robustness clause, and say which one the PAC theorem actually proves.

Hint: The theorem proves the statistical clause under the assumption that samples are independent draws from a fixed distribution; the computational clause concerns the algorithm’s running time, and the robustness clause concerns what happens if the future distribution differs.

A final exercise in judgment: whenever someone hands you a number that claims to describe how many, or a learner that claims to have generalized, ask which parts of the claim are

theorems, which parts are assumptions, and which parts are hope—and state what data, or what reduction, would have to exist to convert each hope into a theorem.

Chapter 3

1990: Alexander A. Razborov

The central habit in this chapter is to prove a universal negative: find a quantity that every allowed gate changes only a little, so every small circuit is trapped in a box the target function can escape.

3.1 An Opening Story: The Proof Game

Three switches guard a vault door, which opens only when all three are on. You build the obvious controller: two AND gates, one combining switches 1 and 2, the second combining that with switch 3. Done. Now a skeptic enters. “You found one circuit,” she says. “Prove that *no* circuit with a single two-input gate can open the vault.”

A single gate reads at most two switches and ignores the third, so the output cannot react to it—the claim is easy. So she makes it harder: five switches, the door opens exactly when at least three are on, and she wants a certificate that no circuit with, say, seven gates can do the job. No single gate ignores enough inputs, and no obvious trick works. This is the *proof game*: one player claims a rule is easy, the other must rule out *every* circuit that might compute it, including

circuits nobody has thought to write down. Alexander Razborov won the 1990 Nevanlinna Prize for winning the hard side of this game for important restricted circuits, proving that natural rules such as “does this graph contain a large clique?” need many gates.

3.2 The Problem

A *Boolean function* is a rule $f: \{0, 1\}^n \rightarrow \{0, 1\}$ reading n bits and answering one. A *Boolean circuit* is a directed acyclic graph whose leaves are the input bits, whose internal nodes are AND, OR, and NOT gates, and whose top node is the output. The *size* is the number of gates; the *depth* is the longest path from an input to the output.

The question is *prove that a specific function requires many gates*. The asymmetry is brutal: an upper bound is one construction, while a lower bound must defeat *all* circuits in a class—an unbounded set of objects, most never seen. Easy to find clever constructions; hard to show cleverness has a limit.

3.3 The World Before

Counting says hard functions exist. There are 2^{2^n} Boolean functions on n bits— $2^{16} = 65,536$ for $n = 4$, about 10^{308} for $n = 10$. A circuit of s gates is described by naming a type and wires per gate, leaving roughly $(cs)^s$ circuits of size s ; for $s = 2^n/n$ this is far smaller than 2^{2^n} . So almost all functions need about $2^n/n$ gates: hard functions are the typical case.

But existence is not an example. The counting argument never names a hard function. “Most functions are hard” and “here is one, with a proof” are different statements, and for decades

nobody could make the second. For explicit functions the best bounds were trivial—roughly enough gates to read the input and nothing more.

The field was rich in upper bounds. Sorting, multiplying, matching: fast algorithms, each a triumph of construction. The discipline could answer “how fast can it be done?” and had almost no tools for “how slow must it be?” The temptation was to judge hardness by cleverness—“I could not find a circuit” is a statement about one person, not a theorem about all circuits.

What was missing: an invariant. A lower bound needs a measure $\mu(f)$ with two properties: each single gate changes μ by a controlled amount, so a circuit of s gates leaves μ inside a controlled range; and the target function sits far outside that range. The creative work is finding a measure that *survives composition of gates*, so composing two gates cannot let it explode.

3.4 The New Idea

Step one: choose a measure every allowed gate changes by a controlled amount. *Step two:* exhibit a target whose measure is unavoidably large. Three moves make this work.

3.4.1 Monotone circuits: remove the eraser

A function is *monotone* if flipping an input bit from 0 to 1 never flips the output from 1 to 0; evidence once collected cannot be forgotten. A *monotone circuit* uses only AND and OR gates, no NOT. This sounds mild but removes the most dangerous power of general circuits: *cancellation*. With NOT, a circuit can combine a fact with its negation, $x \wedge \neg x = 0$, and erase everything it learned. Monotone circuits must keep what they collect, so a measure of “how

much evidence is held” can survive the whole computation.

3.4.2 Polynomial approximations: the stabilizer

Even monotone gates are hard to track exactly. Razborov’s second move replaces each gate’s true behavior by a low-complexity stand-in—a low-degree polynomial (in the original papers, a small family of sets) that agrees with the gate on almost all inputs but is far simpler to count. The whole circuit becomes a low-complexity approximation of the target, and the bound reduces to a comparison: is the target close to *any* object of that low complexity? Approximation is the lever that lets counting arguments reach inside the circuit.

3.4.3 The clique bound: a target that resists

The flagship target: input a graph on n vertices—one bit per possible edge—and ask whether it contains k vertices all pairwise connected, a k -clique. This is monotone: adding edges only creates cliques. Razborov proved any monotone circuit needs $n^{\Omega(\sqrt{k})}$ gates, super-polynomial in n for suitable k . Positive inputs carry huge numbers of incompatible witnesses, while a small circuit keeps only a limited catalogue, so some dense graph with no k -clique slips through. The conceptual shift is the whole lesson: *an upper bound needs one construction; a lower bound must defeat every construction, so find the invariant all small circuits share.*

3.5 How It Works: Worked Examples

3.5.1 No monotone circuit computes parity of three bits

Let p be parity of x_1, x_2, x_3 : output 1 for an odd number of 1s. Then

$$p(0, 1, 0) = 1, \quad p(0, 1, 1) = 0.$$

Flipping x_3 from 0 to 1 moved the output from 1 to 0: an input bit went up and the answer went down. Monotone functions never do that, so parity is not monotone. And every monotone circuit computes a monotone function: input wires are monotone, and AND or OR of monotone functions is monotone. Hence *no* monotone circuit computes parity, of any size. Trivial—yet it teaches the method’s point: monotonicity removes the ability to cancel. Parity needs the circuit to remember that two ones cancel, and monotone hardware has no eraser. Nor is the case moot: reachability, matching, and clique are all monotone, and these are exactly what graph circuits are asked to compute.

3.5.2 AND of three bits: two models, and a measure that fails

First, the model matters. One 3-input AND gate computes $x_1 \wedge x_2 \wedge x_3$ in size 1, depth 1 (unbounded fan-in). Binary gates force a chain, $(x_1 \wedge x_2) \wedge x_3$: size 2, depth 2.

Now try the most naive measure: $\mu(f)$ = number of assignments where f outputs 1. Two tiny circuits show it cannot work. The four-way OR $x_1 \vee x_2 \vee x_3 \vee x_4$ takes three binary gates yet outputs 1 on 15 of 16 assignments: tiny circuit, huge μ . The function $x_1 \wedge (x_2 \vee x_3)$ takes two gates yet outputs 1 on only 3 of 8. Same gate counts, opposite measures. The reason is

composition: for h and g ,

$$\mu(h \vee g) = \mu(h) + \mu(g) - \mu(h \wedge g),$$

and the correction $\mu(h \wedge g)$ is a correlation no gate-local rule can predict. A chain of OR gates can keep μ small by exploiting overlaps, or inflate it by avoiding them. The measure does not respect composition, so it proves nothing—real measures must be *provably stable* under each gate, not merely plausible.

3.6 The Mathematics in Brief

The precise objects first. A *Boolean circuit* with n inputs is a directed acyclic graph whose leaves are the input bits and whose internal nodes are AND, OR, and NOT gates, with one designated output; *size* is the number of gates and *depth* is the longest input-to-output path. A family of circuits whose size grows polynomially in n computes a function in P/poly, the circuit form of the P vs. NP question. Removing NOT gates and keeping only AND/OR gives *monotone circuits*; a *monotone function* satisfies $f(x) \leq f(y)$ whenever $x \leq y$ coordinatewise. Keeping NOT but fixing the depth at a constant and letting gates read unboundedly many inputs gives AC^0 . The headline theorem:

Razborov (1985). Any monotone circuit deciding whether an n -vertex graph contains a k -clique requires $n^{\Omega(\sqrt{k})}$ gates; for $k \approx n^{1/3}$ this outgrows every fixed power of n .

The engine is polynomial approximation. Over the Boolean cube, AND of m inputs is exactly the product $x_1 x_2 \cdots x_m$, and OR is $1 - \prod_{i=1}^m (1 - x_i)$, an exact polynomial of degree m . The key

fact is that the degree can be brought down: OR of m inputs is approximated, pointwise and on average, by a polynomial of degree only $\Theta(\sqrt{m})$ —randomly restricting most variables collapses the OR to a single variable, and $\Theta(\sqrt{m})$ is optimal. Each gate of a small monotone circuit can therefore be replaced by such a low-degree stand-in, and because composing approximations only accumulates error (unlike the naive measure in the worked example, which could explode), a circuit of s gates becomes a function provably close to a polynomial of low degree. If the clique function is far from every such polynomial, no small monotone circuit computes it. Why it’s true: approximants built from few gates must confuse graphs with a genuine k -clique with dense graphs whose largest clique is much smaller, while the clique function must separate exactly those cases.

The general question—an explicit function needing super-polynomial *general* circuits—remains open: negation reintroduces cancellation, and $x \wedge \neg x = 0$ destroys every known measure.

3.7 Limits and Modern Views

Monotone circuits are a restricted model, and the restriction does the work. General circuits can cancel, forget, and coordinate globally; the measures that defeat monotone circuits evaporate at the first NOT gate. Threshold gates—comparing a weighted sum $\sum_i w_i x_i$ to a threshold—add a power none of the classical invariants controls. No one has exhibited an explicit function provably requiring super-polynomial general circuits, and Razborov and Rudich’s *natural proofs* barrier explains part of why: any lower-bound proof of a broad “natural” kind would imply

the nonexistence of certain pseudorandom objects, contradicting widely believed cryptographic conjectures. Many promising techniques cannot resolve P vs. NP; a breakthrough must be unnatural in a precise technical sense.

What survived is the method. The invariant-and-approximant pattern now appears in proof complexity (Razborov’s own work on the pigeonhole principle and bounded arithmetic bounds how concisely some statements can be proved [19]), pseudorandomness, communication complexity, and learning theory, where a “small approximator” is what an efficient learner must find. The 1990 Nevanlinna Prize honored the pattern, not just the clique bound [18]: *to prove a machine cannot be small, find the quantity every small machine must respect, then find a task that violates it.*

3.8 Exercises and Self-Study Guide

- E1.** Compare a balanced formula for AND of $n = 8$ variables with a chain formula. Give the exact size and depth of each.

Hint: Draw both trees: a balanced binary AND tree has depth $\log_2 8 = 3$; a chain has depth 7; both use 7 gates.

- E2.** Prove that the parity function $p(x_1, x_2, x_3)$ is not monotone, and conclude no monotone circuit computes it.

Hint: Find two inputs differing in one bit where p drops from 1 to 0; every AND/OR-only circuit computes a monotone function.

- E3.** Apply a random restriction to the majority-of-three DNF $f = (x_1x_2) \vee (x_2x_3) \vee (x_1x_3)$ by hand:

first set $x_2 = 1$, then $x_2 = 0$. Which terms die, and what function remains?

Hint: Setting $x_2 = 1$ turns x_1x_2 and x_2x_3 into x_1 and x_3 ; setting $x_2 = 0$ kills all three terms.

- E4.** Explain why a lower bound for monotone circuits says nothing directly about general circuits, using AND of three bits as a test case.

Hint: The same function is cheaper in a stronger model: one 3-input AND beats any binary-circuit bound, and a NOT gate adds powers monotone circuits lack.

- E5.** Compute $\mu(f)$, the number of assignments where f outputs 1, for $x_1 \vee x_2 \vee x_3 \vee x_4$ and for $x_1 \wedge (x_2 \vee x_3)$. Why do these numbers rule out “small circuit implies small μ ”?

Hint: The four-way OR outputs 1 on 15 of 16 assignments; the two-gate function on 3 of 8. Same gate counts, opposite measures.

- E6.** Prove that no circuit with a single two-input AND, OR, or NOT gate computes majority of three bits.

Hint: A one-gate circuit ignores some input x_j ; pick two inputs differing only in x_j where majority flips, e.g. $(0, 1, 0)$ vs. $(0, 1, 1)$.

- E7.** Read a published lower-bound proof (Razborov’s monotone clique bound, or an exposition of it) and mark every place the machine model is used.

Hint: Ask the four questions: what is the model, what quantity is small for every small machine, why does it survive each gate, where does the target violate it.

A final exercise in judgment: whenever you read that a circuit class cannot compute a function, name the forbidden power—cancellation, global coordination, thresholds—and ask which future gadget would make the old measure fail. Naming the next forbidden power is how lower bounds progress.

Chapter 4

1994:Avi Wigderson

*The central habit in this chapter is to let a verifier's random challenge choose what to inspect,
and to judge a claimant by the probability of being caught rather than by any single answer.*

4.1 An Opening Story: Two Drawings and a Know-It-All

Two cards lie on a table, each showing a drawing of five dots and five lines: one a ring, the other a star. Sam claims he can always tell them apart, even after someone renames the dots.

The teacher tests him. She secretly picks a card, renames all five dots with a fresh shuffle each round, shows the result, and asks: ring or star? If the drawings really differ, Sam answers correctly every round. If they secretly showed the same shape, the rename hides which card was picked and even a perfect answerer is right only half the time; ten rounds of guessing pass with probability $(1/2)^{10} < 1/1000$.

Nothing was checked directly. The teacher asked a question she could not answer herself, and the shuffle made it fair: a challenge known in advance could be met with a prepared lie.

Randomness turned “these two are not the same”—a claim with no short written proof—into tests a liar cannot all pass.

4.2 The Problem

One person, the *prover*, claims a fact and will do enormous work to defend it. A second, the *verifier*, has little time and must decide whether to believe it. The prover might lie. How can the verifier be convinced?

The classical answer is a *certificate*: a fixed string a fast deterministic algorithm checks. For “these two drawings are the same shape” the certificate is easy: hand over the renaming and compare. The opposite claim, “these two are *not* the same shape,” has no known short certificate. For graphs G_1, G_2 this is graph non-isomorphism (GNI): an isomorphism witnesses that they *are* the same, but no short static certificate of non-isomorphism is known.

The verifier cannot recompute, cannot trust, and cannot peek inside the prover’s head. The escape is conversation: questions the prover could not have anticipated. *Can a weak verifier be convinced of a statement with no short static certificate, by a powerful prover who might be lying?*

4.3 The World Before

For most of the history of logic, a proof was a static object: a certificate string checked mechanically. Computer science inherited this as the class NP: every yes-instance has a short certificate a deterministic verifier accepts, and no no-instance has one. “Checking is easy, finding

is hard” was the whole vocabulary.

GNI was an embarrassment for that picture. Its yes-instances have short certificates; its no-instances do not. Since a certificate can only witness membership, “not isomorphic” looked like a problem with no proof at all.

Randomness made things worse before better: randomized algorithms were viewed as a confession of weakness, and the goal was to remove the coin flips. Nobody asked whether a verifier could *use* coin flips as a resource.

What was missing was a proof as a conversation with two extra resources: *interaction* (follow-up questions) and *public randomness* (unpredictable coins). Without both, GNI sat outside any efficient verification framework.

4.4 The New Idea

4.4.1 Conversation, not certificate

An *interactive proof* replaces the certificate with a dialogue. The prover may compute anything; the verifier flips coins, asks questions, and the prover’s answer is conditioned on a challenge not known in advance. A pre-announced challenge could be met by one fixed lie; the random challenge is the whole point.

4.4.2 The two promises

Completeness: if the claim is true, an honest prover convinces the verifier with high probability.

Soundness: if it is false, no prover, however clever, convinces it except with small probability.

Both probabilities are over the verifier's coins, not the prover's goodwill. Repetition with fresh coins multiplies the error: $1/2$ becomes $(1/2)^k$, below $1/1000$ at $k = 10$.

4.4.3 Arithmetization: logic becomes algebra

Rewrite a Boolean computation as polynomial equations over a finite field. A bit x is forced by $x^2 - x = 0$; AND becomes multiplication ($x \wedge y \mapsto xy$) and OR becomes $x + y - xy$. A circuit becomes one polynomial identity, checked at a few random points.

4.4.4 $\text{IP} = \text{PSPACE}$

Lund, Fortnow, Karloff, Nisan, and Shamir proved that every problem solvable with polynomial memory—the class PSPACE —has an interactive proof. A conversation is as powerful as any algorithm for the problem.

4.4.5 Zero knowledge: validity without the secret

Wigderson's signature addition: prove a statement true while revealing nothing about why. A zero-knowledge proof is complete and sound and has transcripts that can be simulated without the witness; if the verifier could have produced the conversation alone, it carries no information.

A proof is not an object; it is a conversation.

4.5 How It Works: Worked Examples

4.5.1 Graph non-isomorphism, round by round

Let G_1 be the ring, G_2 the star—five vertices and five edges each, not isomorphic. One round: the verifier flips a coin, applies a uniformly random relabeling π to the chosen graph, and sends it to the prover, who answers “ring” or “star.”

If the claim is true, the honest prover always identifies the graph and passes with probability 1. If it is false—the graphs *are* isomorphic—a relabeled G_1 and a relabeled G_2 come from exactly the same distribution, so the answer is independent of the coin and correct with probability $1/2$; ten rounds leave error at most $(1/2)^{10} < 1/1000$. The coins matter: a fixed rename could be answered by a memorized lie; a fresh one cannot.

4.5.2 Checking a polynomial identity

The prover claims $x^2 = 1$ for every x . Over $\mathbb{F}_{101}$, the verifier picks a random $r \in \{0, \dots, 100\}$, computes r^2 and 1, and checks equality. The difference $x^2 - 1 = (x - 1)(x + 1)$ has exactly two roots, $x = 1$ and $x = 100$, so a random point exposes the lie with probability at most $2/101 \approx 2\%$. Three independent checks multiply the error to about $(2/101)^3 < 10^{-5}$.

This is the engine of arithmetization: two distinct polynomials of degree at most d over a field of size q agree at at most d points, so one random evaluation gives meaningful evidence.

4.5.3 Zero knowledge on a small graph

A triangle a, b, c is colored red, blue, red. The prover wants to prove the triangle is 3-colorable without naming a color. Each round it picks a fresh random permutation of the colors, commits to every vertex's color (hidden until opened), and the verifier picks one of the three edges and asks to open just its two endpoints. The check: the colors differ.

A proper coloring passes every round. A lying prover with a bad edge is caught exactly when that edge is picked, with probability at least $1/3$ per round; k rounds leave error at most $(2/3)^k$, about 1.7% at $k = 10$. Nothing leaks because each round uses a fresh permutation: the opened pair is always just “two distinct colors from three,” exactly what a simulator without the coloring can reproduce.

4.6 The Mathematics in Brief

An interactive proof for a language L is a pair of machines exchanging messages. For an instance x :

$$x \in L \Rightarrow \mathbb{P}[V \text{ accepts with honest } P] \geq 2/3, \quad x \notin L \Rightarrow \mathbb{P}[V \text{ accepts with any } P] \leq 1/3.$$

The gap between $2/3$ and $1/3$ is deliberate: repetition pushes the $1/3$ toward 0 exponentially while an honest prover stays accepted. Probabilities are over the verifier's coins only.

The workhorse lemma: *two distinct polynomials of degree at most d over a field of size q agree at at most d points*—their difference is a nonzero polynomial with at most as many roots as its degree.

Arithmetization rests on three exact translations (field of characteristic not 2):

$$x \in \{0, 1\} \iff x^2 - x = 0, \quad x \wedge y \iff xy, \quad x \vee y \iff x + y - xy.$$

A circuit becomes a system of polynomial equations; a claimed computation becomes a polynomial identity. Lund–Fortnow–Karloff–Nisan and Shamir then proved $\text{IP} = \text{PSPACE}$: interactive proofs capture exactly polynomial memory. Constructively: unwind the computation into a layered circuit, arithmetize it, and verify by a sum-check that justifies a multivariate sum one variable at a time.

Zero knowledge adds a simulation condition: a polynomial-time machine that, given only $x \in L$, produces transcripts indistinguishable from real ones; since it needs no witness, real transcripts cannot be milked for one. The costs are asymmetric: the prover may work exponentially, the verifier polynomially, and soundness error and randomness spent are part of the resource bill.

4.7 Limits and Modern Views

The costs are real: for GNI the prover must actually decide isomorphism, and for arithmetized computations it must manipulate polynomials the verifier never touches. The guarantee is probabilistic, not certain. Zero knowledge adds friction: commitments rest on computational assumptions, transcripts must be rechecked for leakage, and reusing randomness across rounds can destroy privacy.

The ideas outgrew their origin in three directions. *Probabilistically checkable proofs* (PCPs) showed an ordinary NP certificate can be checked by reading a handful of bits—a static object

with interactive power. *Succinct proofs* (SNARKs, STARKs) let a cloud computer convince a laptop that a huge computation ran correctly, trading prover time against tiny proofs, amid debates about trusted setup and proof length. *Post-quantum cryptography* asks whether the assumptions behind commitments and succinct proofs survive a quantum adversary, and answers with lattice problems [20, 21]. The lessons persist: expose algebraic structure, sample random constraints instead of rechecking everything, and separate validity from secrecy.

4.8 Exercises and Self-Study Guide

E1. Write the polynomial constraint that forces a variable to be a bit, and verify it in $\mathbb{F}_3$ at

$$x = 0, 1, 2.$$

Hint: $x^2 - x = 0$; $x = 2$ gives $4 - 2 = 2 \neq 0$, and note why the trick silently fails in $\mathbb{F}_2$.

E2. Show that $P(x) = x^2$ and $Q(x) = 1$ agree at exactly two points in $\mathbb{F}_7$, and list them.

Hint: Solve $x^2 = 1$ over $\{0, \dots, 6\}$; the solutions are $x = 1$ and $x = 6$, so a random point catches the difference with probability $5/7$.

E3. A protocol has soundness error $1/3$. Find the smallest k such that k independent rounds bring the error below $1/100$.

Hint: $(1/3)^4 = 1/81$ is still above $1/100$; $k = 5$ gives $1/243$.

E4. Construct a zero-knowledge transcript that accidentally reveals the witness, using the 3-coloring protocol on the triangle.

Hint: Reuse the same color permutation in two rounds opening two edges that share a vertex; combining the opened colors pins down the full coloring.

E5. Compute the soundness error of the GNI protocol for $k = 5$ rounds, and state the honest prover's

acceptance probability.

Hint: $(1/2)^5 = 1/32 \approx 3\%$; the honest prover is still accepted with probability 1.

- E6.** Convert the Boolean formula $(x \wedge y) \vee z$ into polynomial equations, including bit constraints.

Hint: Introduce $a = xy$ for the AND; the OR becomes $a + z - az$; each variable satisfies $t^2 - t = 0$.

- E7.** Explain why the verifier must not reveal its random challenge before the prover answers, using GNI.

Hint: A pre-announced rename is a single test with a memorized answer; randomness makes every round a fresh test.

A final exercise in judgment: before trusting any verification system, name what the verifier's random coins protect against. If the answer is "a prepared lie," find the challenge the prover cannot predict; if you cannot name it, the protocol is only a certificate wearing a costume.

Chapter 5

1998: Peter W. Shor

The central habit in this chapter is to hunt for a hidden period, and to ask how interference can make a repeating pattern measurable when sampling one value at a time is too slow.

5.1 An Opening Story: The Wheel and the Strobe Light

A black wheel with one red dot spins at an unknown rate. Flash a strobe once per second: at the wrong rate the dot darts to a different spot each flash; in time with the wheel it appears frozen. One sample per flash, yet the period is known exactly.

The same trick works when a process repeats every r steps and you may sample it once per step: taken one at a time, the samples look arbitrary. The strobe combines many at once, so combinations agreeing with the true period *add up* and everything else blurs away. Reinforce the compatible, cancel the incompatible—that is the whole mechanism.

In 1998 Peter W. Shor received the Rolf Nevanlinna Prize for building a computer from exactly this mechanism: factoring hides a repeating pattern, and a quantum computer makes it

visible and, with the period, obtains the factors.

5.2 The Problem

Factoring. Multiplying is fast: $23 \times 41 = 943$; reversing it is not. Trial division works for small numbers, but for an n -bit N , trying all primes up to $\sqrt{N}$ costs about $2^{n/2}$ steps—astronomical at $n = 200$. The best classical methods (Pollard’s rho, the quadratic sieve, the number field sieve) are still not polynomial: the number field sieve needs roughly $\exp(cn^{1/3})$ steps. Nobody has proved factoring hard; the belief rests on fifty years of failed attempts.

Why it matters. Public-key cryptography lives on this asymmetry. In RSA the public key is a product $N = pq$; anyone can encrypt, but only someone knowing p and q can decrypt. The related *discrete logarithm* problem—find x given $g^x \equiv h \pmod{N}$ —protects key exchange and signatures. Both bet that reversing an easy computation is hard.

The physics, in one paragraph. A qubit is a superposition $\alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$; amplitudes can add and cancel, but measuring samples one outcome—0 with probability $|\alpha|^2$ —and only that survives. Is there a problem whose difficulty comes from sampling one thing at a time, that a machine built to interfere amplitudes could solve instead?

5.3 The World Before

Factoring grew harder to do as it grew more useful: trial division covers a few digits, Pollard’s rho tens of digits, the quadratic sieve reached 100 digits in 1991, and the number field sieve passed 250 digits in the 2020s. All remain subexponential, and RSA keys grew from 512 bits to

2048 because the multiply-versus-factor gap seemed safe.

Quantum computing had a quieter history. Feynman (1982) suggested a quantum machine might be the only efficient way to *simulate* quantum physics; Deutsch (1985) and Deutsch–Jozsa (1992) separated quantum from classical machines only on problems with no practical use. As of the early 1990s no quantum algorithm beat the best classical one on any realistic task, and many doubted a useful quantum algorithm existed.

The doubt was grounded: n qubits represent 2^n inputs at once, but measure and you get one random outcome with all structure discarded. *Parallelism alone is not power*. Missing was a way to process amplitudes *before* measuring—letting them add and cancel so a meaningful statistic survives.

5.4 The New Idea

5.4.1 Factoring becomes a hidden period

The pivot is classical number theory: *factoring reduces to finding a period*. Choose a sharing no factor with N (say $a = 7$, $N = 15$). The powers $a^1, a^2, a^3, \dots$ modulo N take finitely many values, so they repeat; since a and N share no factor, the first repeated value is a^1 itself. The sequence cycles with period r , the least positive integer with $a^r \equiv 1 \pmod{N}$, called the *order* of a modulo N . Computing powers until the pattern returns—the classical way to find r —is the slow, sample-by-sample search the strobe defeats.

What makes r valuable: if r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then

$$\gcd(a^{r/2} - 1, N) \quad \text{and} \quad \gcd(a^{r/2} + 1, N)$$

are genuine factors of N . Indeed N divides $(a^{r/2} - 1)(a^{r/2} + 1)$ because it divides $a^r - 1$, yet neither factor alone—otherwise the order would be smaller, or $a^{r/2} \equiv -1$ —so each gcd pulls out a proper divisor. Finding the factors *is* finding the period.

5.4.2 Quantum period-finding: interference does the searching

Before Shor, a computation was a step-by-step pipeline from input to output. The new idea: compute $a^x \bmod N$ for many x in superposition, then apply an operation that converts *periodic* input into *concentrated* output—the quantum Fourier transform (QFT). A classical Fourier transform already has the strobe’s property: a signal of period r concentrates its energy in a few frequencies; a random signal spreads it. The quantum version does this to amplitudes, which can interfere: outcomes agreeing with the hidden period reinforce, others cancel. A measurement then lands, with high probability, on a frequency carrying the period; continued fractions recover r , and the number theory above turns it into factors. The speedup is not “trying every answer at once”: interference moves probability toward answers agreeing with the period and silences the rest.

5.4.3 Quantum error correction: protecting the fragile

A long computation faces a second enemy: the state decays. Classical error correction copies bits and compares—but quantum states cannot be copied, and measuring a qubit collapses it.

Shor's solution spreads one *logical* qubit across several *physical* ones and measures carefully chosen combinations whose values reveal which error occurred without revealing the information itself. Redundancy plus a syndrome, arranged so checking does not disturb what it protects. Below a physical error threshold, arbitrarily long computations become possible in principle.

5.5 How It Works: Worked Examples

5.5.1 Factoring 15 by order-finding

Factor $N = 15$ by finding the order of $a = 7$. First, $\gcd(7, 15) = 1$. Compute powers modulo 15:

$$7^1 = 7,$$

$$7^2 = 49 \equiv 4 \pmod{15} \qquad (49 - 45 = 4),$$

$$7^3 = 4 \cdot 7 = 28 \equiv 13 \pmod{15} \qquad (28 - 15 = 13),$$

$$7^4 = 13 \cdot 7 = 91 \equiv 1 \pmod{15} \qquad (91 - 90 = 1).$$

The first power equal to 1 is the fourth, so the order is $r = 4$.

Since $r = 4$ is even, $a^{r/2} = 7^2 = 49 \equiv 4 \pmod{15}$, and $4 \not\equiv -1 \pmod{15}$, so the formula applies:

$$\gcd(7^2 - 1, 15) = \gcd(48, 15) = 3, \qquad \gcd(7^2 + 1, 15) = \gcd(50, 15) = 5.$$

The factors appear directly: $15 = 3 \cdot 5$.

The quantum part supplies $r = 4$ without the four-step search: its output is a frequency, and continued fractions recover the denominator 4. The algorithm is probabilistic in two ways: the

order may be odd, or $a^{r/2}$ may be -1 —then the formula fails and one retries with a different a ; and the measured frequency only approximates a fraction with denominator r , so a few runs suffice. The final arithmetic is exact; only the search is probabilistic.

5.5.2 A strobe in frequency space: the Fourier transform of a repeating pattern

Take the length-four signal $[1, 0, 1, 0]$, period 2. Its discrete Fourier transform is $X_k = x_0 + x_1\omega^k + x_2\omega^{2k} + x_3\omega^{3k}$, where $\omega = e^{-2\pi i/4}$ is a quarter-turn, so $\omega^2 = -1$ and $\omega^4 = 1$:

$$X_0 = 1 + 0 + 1 + 0 = 2,$$

$$X_1 = 1 + 1 \cdot \omega^2 = 1 - 1 = 0,$$

$$X_2 = 1 + 1 \cdot \omega^4 = 1 + 1 = 2,$$

$$X_3 = 1 + 1 \cdot \omega^6 = 1 - 1 = 0.$$

The transform is $(2, 0, 2, 0)$: two peaks, two exact zeros. Repetition has become concentration, exactly as the strobe freezes a wheel; all the mass sits at frequency 0, the constant part, and frequency 2, the rate of repetition.

Contrast the non-repeating signal $[1, 1, 1, 0]$: its transform is $X_0 = 3$, $X_1 = -i$, $X_2 = 1$, $X_3 = i$, magnitudes $(3, 1, 1, 1)$ —energy spread across every frequency. In the periodic case the entries agreeing with the repetition add up and the others cancel to zero. The quantum Fourier transform performs this interference on amplitudes, so a single measurement sees the peak and not the noise.

5.6 The Mathematics in Brief

A qubit is a unit vector $\alpha|0\rangle + \beta|1\rangle$ with $|\alpha|^2 + |\beta|^2 = 1$; measuring samples 0 or 1 with those probabilities and collapses the state. Several qubits carry 2^n coefficients, but an algorithm may touch them only through unitary operations and measurements.

Order. For $\gcd(a, N) = 1$, the *order* r is the least positive integer with $a^r \equiv 1 \pmod{N}$; the sequence $a^x \bmod N$ repeats with period r .

The factor-recovery identity. If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then $\gcd(a^{r/2} - 1, N)$ and $\gcd(a^{r/2} + 1, N)$ are nontrivial factors of N . *Why it is true:* N divides $(a^{r/2} - 1)(a^{r/2} + 1)$ yet divides neither factor, so each gcd lies strictly between 1 and N .

The quantum Fourier transform. A signal of period r has a Fourier transform concentrated at frequencies whose denominators divide r . *Why it is true:* values spaced r apart carry identical phase and add; the others spread around the circle and cancel. Shor's algorithm superposes $a^x \bmod N$, applies the QFT, and measures; continued fractions recover r from the measured frequency.

Error correction. One logical qubit is encoded across many physical qubits, and syndrome measurements reveal errors without revealing the logical state. The threshold theorem: below a physical error rate determined by the code and noise model, arbitrarily long computations become reliable with modest overhead; above it, adding qubits can make things worse.

Where the speed comes from. The gain is interference, not parallelism: the machine never reads every answer; it arranges amplitudes so answers compatible with the hidden period keep

their probability and incompatible answers lose it, then measures once.

5.7 Limits and Modern Views

Current devices are small and noisy—a few hundred to a little over a thousand physical qubits, far below the millions that resource estimates place behind a single cryptographically relevant factorization. The threshold theorem is a statement about a noise model, not a guarantee about any chip; correlated errors and imperfect operations can violate its assumptions. The theorem, the resource estimate, and the engineering forecast are three different claims; the theory establishes the first, while the third remains open [22,23].

What is threatened, precisely? Systems whose security rests on factoring or discrete logarithms, such as RSA and conventional Diffie–Hellman or elliptic-curve key exchange. Symmetric cryptography (AES, SHA) is different: the best generic quantum attack, Grover’s search, only squares the effective security, so doubling the key length restores it. Post-quantum cryptography seeks replacements believed hard even for quantum machines—lattices, error-correcting codes, hash functions, isogenies. None is unconditionally safe; each is a bet on an assumption.

5.8 Exercises and Self-Study Guide

E1. Verify the factor-recovery formula on another pair: take $N = 21$, $a = 2$. Compute the powers of 2 modulo 21, find the order, and use the gcd identity to factor 21.

Hint: Follow the 15 example line by line. The order is 6, $2^3 = 8 \not\equiv -1 \pmod{21}$, and the two gcds are 7 and 3.

- E2.** Compute the powers of $a = 14$ modulo 15, find its order, and apply the gcd identity. Explain why this a is a poor choice for the algorithm.

Hint: The order is 2 and $a^{r/2} = 14 \equiv -1 \pmod{15}$; the two gcds come out 1 and 15, not proper factors.

- E3.** Explain why measuring a qubit destroys access to its amplitudes. Give the state $\alpha|0\rangle + \beta|1\rangle$ and describe what happens to α and β after one measurement.

Hint: After the measurement the state is $|0\rangle$ or $|1\rangle$; one amplitude has vanished. Write the probabilities of the two outcomes before the collapse.

- E4.** Explain why superposition alone is useless, using the coin metaphor: n qubits represent 2^n values, yet a measurement returns one random outcome.

Hint: Unless amplitudes were engineered beforehand, the chance that a single sample hits a useful answer is exponentially small.

- E5.** Compute the discrete Fourier transform of the length-eight signal $[1, 0, 1, 0, 1, 0, 1, 0]$ and state where its peaks sit.

Hint: All nonzero entries lie at even positions, so each X_k is a sum of four unit vectors spaced a quarter-turn apart; they add to 4 only at $k = 0$ and $k = 4$.

- E6.** List every assumption hidden in a threshold-theorem statement, and identify which one an engineer could measure on a real device.

Hint: Errors are assumed independent or local; gates and measurements must be accurate enough; correction must run fast enough. The effective per-operation error rate is what to measure.

- E7.** Separate cryptographic primitives into those threatened by order-finding and those affected only by generic quantum search. Justify each placement.

Hint: Order-finding attacks anything built on modular exponentiation, such as RSA and discrete-logarithm key exchange; Grover's search only halves the effective key length of a hash or block cipher.

A final exercise in judgment: before celebrating any quantum speedup, ask what structure the interference is amplifying. If you cannot name the hidden period—the pattern the amplitudes are agreeing on—then the advantage, like the strobe's frozen wheel, may be an illusion of coherence rather than a real discovery.

Chapter 6

2002:Madhu Sudan

The central habit in this chapter is to treat ambiguity as information: when the noise is heavy, the honest output of a decoder is a short list of every message that could explain what arrived, rather than one forced guess.

6.1 An Opening Story: Two Garbled Words, Four Plausible Messages

A friend texts you a four-word sentence, and the channel scrambles two of the four words. What arrives is

MEET ?? OLD ??

the question marks are unreadable words. The sentence could have been “MEET AT OLD DOCK,” “MEET IN OLD TOWN,” or “MEET BY OLD MILL”—each explains exactly the fragments you received. A machine forced to print one sentence must guess—a coin flip that

may send you to the wrong place. Two ruined words out of four outrun what cleverness can resolve from this message alone.

Yet the receiver is not helpless: the plausible sentences form a short list, and the true one is on it. A second text—“bring the fishing rod”—narrows the two together to one winner. The first message did not need to decide; it only had to keep every candidate alive until better evidence arrived. That division of labour, *report the ambiguity now, resolve it later*, is the subject of this chapter, and Madhu Sudan’s 2002 Rolf Nevanlinna Prize recognized the mathematics that made it efficient.

6.2 The Problem

An error-correcting code expands a message with redundancy so that noise can be repaired. A codeword is a valid expanded message; *decoding* recovers the message from the corrupted string.

The classical contract is strict. If the code has *minimum distance* D —the fewest positions in which two codewords differ—a decoder is guaranteed success exactly when the number of corrupted positions is at most $(D - 1)/2$: inside that radius the received word is closer to its true codeword than to any other. This is *unique decoding*: one received word, one answer, always right, provided the noise is mild.

Beyond radius $(D - 1)/2$ the contract collapses: two codewords can be equidistant from the received word, so no rule always picks the true one. A unique decoder then guesses, and a guess destroys information. The hard question: must decoding stop there, or was the *output contract* wrong?

6.3 The World Before

In 1950 Richard Hamming set the modern framework: a code is a set of strings with minimum distance D , and a decoder corrects up to $(D - 1)/2$ errors by returning the nearest codeword. Hamming also priced codes—longer codewords buy more protection—so the subject could ask quantitative questions.

Reed–Solomon codes (1960) gave the framework its most beautiful example. Write the message as a polynomial $p(x)$ of degree at most k ; the codeword is its values $p(a_1), \dots, p(a_n)$ at n fixed points. Two distinct such polynomials agree in at most k positions, so the minimum distance is $D = n - k$. Algebraic algorithms—Berlekamp–Welch, Berlekamp–Massey—decoded these codes in polynomial time all the way to the $(D - 1)/2$ bound.

The barrier looked fundamental. A received word can sit halfway between two codewords, so no unique answer is always correct beyond half the minimum distance; this was read as “heavy noise is hopeless.” True for unique decoding, false for a different notion of “the answer.” In the late 1950s Elias and Wozencraft independently proposed *list decoding*: return every codeword within the radius, not just one. The geometry was clear, but no efficient algorithm existed—trying every message is exponential—so it stayed exotic for thirty-five years.

One well-known fact was the seed: two distinct polynomials of degree at most k agree in at most k points, so agreement is sharp evidence. But organizing the search over all messages without checking them one by one—that nobody saw.

6.4 The New Idea

Sudan's insight begins with a question about what an answer should be. When several messages genuinely explain the received word, returning one candidate is not solving the problem; it hides the fact that the problem has several solutions. The correct output is the set of all explanations.

Ambiguity can be managed mathematically: a short list is a better answer than a forced guess.

The surprise is that this is not just philosophy; it makes heavy noise tractable. Sudan's 1997 algorithm rests on two counting arguments that expose every plausible message at once.

Unknowns versus equations. Seek a nonzero bivariate polynomial $Q(x, y)$ vanishing at every received point (a_i, b_i) : one linear equation per point, in Q 's unknown coefficients. More coefficients than points means the system is under-determined, so a nonzero solution must exist. Two variables are essential: a univariate $Q(x)$ vanishing at n points needs degree at least n , while a bivariate $Q(x, y)$ of small total degree has far more coefficients than n .

Roots versus degree. If a message p agrees with the received word in t positions, $Q(x, p(x))$ is a univariate polynomial with t roots. When t exceeds its degree, $Q(x, p(x)) \equiv 0$, so $(y - p(x))$ divides Q : *every sufficiently plausible message is a factor of Q* . Factoring Q prints the list, which is short: at most one factor per degree of y .

Both counts succeed together. Sudan's algorithm then decodes in polynomial time with about $n - 2\sqrt{nk}$ errors, and Guruswami–Sudan with about $n - \sqrt{nk}$; the unique-decoding radius is only about $(n - k)/2$. For small k the list-decoding radii dwarf it, and the only price is a short list. The errors once declared “impossible” became routine.

6.5 How It Works: Worked Examples

6.5.1 Four symbols, two errors, six honest answers

Take a Reed–Solomon code with message polynomial $p(x) = 2x + 1$ of degree $k = 1$, evaluated at $x = 0, 1, 2, 3$:

$$(p(0), p(1), p(2), p(3)) = (1, 3, 5, 7).$$

The minimum distance is $D = n - k = 3$, so unique decoding corrects at most $(D - 1)/2 = 1$ error. Suppose the received word is

$$(1, 0, 0, 7),$$

the second and third symbols (values 3 and 5) corrupted. Two errors: unique decoding is beyond its guarantee.

List decoding returns every degree-1 polynomial agreeing with the received word in at least two positions. Since two distinct degree-1 polynomials agree in at most one point, each candidate is identified by the pair of positions where it agrees. From the points $(0, 1), (1, 0), (2, 0), (3, 7)$:

- $(0, 1)$ and $(3, 7)$: slope $(7 - 1)/(3 - 0) = 2$, so $p(x) = 2x + 1$ —the true message.
- $(0, 1)$ and $(1, 0)$: slope -1 , so $p(x) = 1 - x$.
- $(1, 0)$ and $(3, 7)$: slope $7/2$, so $p(x) = \frac{7}{2}(x - 1)$.

Three more pairs give $p(x) = 1 - \frac{1}{2}x$, $p(x) = 0$, and $p(x) = 7(x - 2)$, for six candidates in total.

Check one: $p(0) = 1$, $p(1) = 0$, matching the received values. Six different messages explain the same evidence, the true one among them. A unique decoder might pick $p = 1 - x$ and lose

the message; the list keeps $2x + 1$ alive until later evidence selects it. (Reals are used for familiar arithmetic; over any field with at least four evaluation points the story is identical.)

6.5.2 The bivariate polynomial in action

Now use Sudan's machinery on the same points $(0, 1), (1, 0), (2, 0), (3, 7)$. A bivariate polynomial of total degree at most 2 has 6 coefficients:

$$Q(x, y) = c_0 + c_1x + c_2y + c_3x^2 + c_4xy + c_5y^2.$$

Requiring $Q(a_i, b_i) = 0$ at the four points gives four homogeneous linear equations in six unknowns, so a nonzero solution must exist. One is

$$Q(x, y) = 7x^2 + 2y^2 - 21x - 16y + 14.$$

Check: at $(0, 1)$, $0 + 2 - 0 - 16 + 14 = 0$; at $(1, 0)$, $7 + 0 - 21 + 0 + 14 = 0$; the other two cancel the same way. *More unknown coefficients than constraints forces a nonzero Q* , and finding it is routine linear algebra.

Now substitute a candidate. For the true message $p = 2x + 1$,

$$Q(x, 2x + 1) = 15x^2 - 45x = 15x(x - 3),$$

with roots $x = 0, 3$, exactly its agreement positions. For $p = 1 - x$,

$$Q(x, 1 - x) = 9x^2 - 9x = 9x(x - 1),$$

roots at $x = 0, 1$. Here agreement $t = 2$ equals the degree of $Q(x, p(x))$, so roots appear but are not forced; the forcing step needs the counting tilted. In Sudan's real regime—many points,

$k \ll n$ —a genuine candidate agrees in t positions *exceeding* that small degree; $Q(x, p(x))$ then has more roots than its degree permits, vanishes identically, and the candidate is a factor of Q . Factoring Q , or solving for its y -roots, prints the whole list in polynomial time.

6.6 The Mathematics in Brief

The agreement lemma. Two distinct polynomials of degree at most k agree in at most k points.

Why: their difference is a nonzero polynomial of degree at most k , which has at most k roots.

Reed–Solomon geometry. A message is a polynomial of degree at most k ; its codeword is its n values. Hence $D = n - k$ and the unique radius is $(D - 1)/2 = (n - k - 1)/2$.

The list-decoding radius. A Johnson-type bound says that for any code of distance $D = n - k$, agreement radius $\sqrt{nk}$ (about $n - \sqrt{nk}$ errors) yields only a polynomially bounded list. Sudan achieves this up to a constant factor $(n - 2\sqrt{nk})$; Guruswami–Sudan reaches the Johnson bound exactly. The formula $\sqrt{nk}$ balances the counting: enough points for a sparse Q , yet enough agreements for t to outrun the degree of $Q(x, p(x))$.

The $Q(x, y)$ method. Choose Q with more coefficients than points, so $Q(a_i, b_i) = 0$ has a nonzero solution. A candidate agreeing in t positions makes $Q(x, p(x))$ a degree- $(< t)$ polynomial with t roots; it must vanish identically, so p divides Q . With B bounding the y -degree, the list has at most B members. Every inequality has an algorithmic meaning: existence from linear algebra, extraction from factoring.

The proof-checking connection. The same “local evidence forces a global conclusion” pattern powers probabilistically checkable proofs: encode a long proof, test a local relation at a few

random positions, and let distance make a wrong proof fail most tests. Decoding and proof-checking are two faces of one idea: *redundancy turns a few observations into a guarantee about the whole object.*

6.7 Limits and Modern Views

The list is small but not free: its size grows like $\sqrt{n/k}$, and the field must supply n distinct evaluation points, limiting block length; algebraic-geometry codes relax this and give the best known length–rate–distance tradeoffs. The Guruswami–Sudan refinement—weights and multiplicities in the interpolation—reaches the Johnson bound and extends to *soft-decision decoding*, where each symbol carries a reliability score and weights favour trustworthy positions.

The PCP connection went beyond metaphor. These algebraic verification ideas became the engine of the PCP theorem—every proof can be encoded so that a verifier reading a constant number of random positions separates valid proofs from almost all invalid ones—and from it came hardness of approximation: problems like MAX-3SAT are hard even to approximate [24,25]. Engineering inherited the same ideas: erasure coding (Reed–Solomon’s descendant) protects RAID arrays and data centres, and list decoding appears in regenerating codes that repair a failed node from a few surviving blocks.

List decoding remains the honest statement of what decoding means. Heavy noise is not “unsolvable”; it produces a short list, and shortening it to a winner is a job for side information, not the decoder. The field’s slogan changed from “correct the errors” to “know the plausible messages.”

6.8 Exercises and Self-Study Guide

- E1.** Prove the agreement lemma: two distinct polynomials of degree at most k agree in at most k points.

Hint: Subtract the two polynomials; a nonzero polynomial of degree at most k has at most k roots.

- E2.** Verify that $(1, 0, 0, 7)$ is at Hamming distance 2 from the codewords of both $p = 2x + 1$ and $p = 1 - x$, while the unique-decoding radius is 1. Why must a unique decoder guess?

Hint: Write both codewords and count differing positions; a distance-3 code cannot tell them apart at radius 2.

- E3.** Set up $Q(x, y) = c_0 + c_1x + c_2y + c_3xy$ vanishing at the four received points and show it has only the zero solution; explain why adding x^2 and y^2 terms yields a nonzero one.

Hint: The first three points force $c_0 = c_1 = c_2 = 0$, the fourth forces $c_3 = 0$: four unknowns, four equations, no slack.

- E4.** A Reed–Solomon code has $n = 100$ points and message degree $k = 5$. Compare the unique-decoding radius with the list-decoding radius and state the agreement threshold for a guaranteed place on the list.

Hint: Unique radius $(n - k - 1)/2 \approx 47$; list radius about $100 - \sqrt{500} \approx 78$ errors; the Johnson threshold is roughly $t > \sqrt{nk} \approx 22$.

- E5.** Explain why Q must have two variables; what goes wrong with a univariate $Q(x)$?

Hint: To vanish at n points a univariate Q needs degree at least n , so coefficients never beat the equations; the second variable carries the received value.

- E6.** The repetition code maps $0 \mapsto 000$, $1 \mapsto 111$. A local test reads two random positions and accepts if they match. Show it never rejects a valid codeword, and compute the rejection probability for 001.

Hint: For 001 the pairs $(0, 0)$, $(0, 1)$, $(0, 1)$ disagree in two of three cases.

- E7.** Give an example where the worked-example list collapses to one message only after a second constraint, and name the constraint.

Hint: The received word cannot separate $2x + 1$ from $1 - x$; a second transmission revealing $p(1) = 3$ leaves only $2x + 1$.

A final exercise in judgment: whenever a decoder is forced to return one answer, ask whether the ambiguity it suppresses is real. If two messages genuinely explain the same evidence, the honest output is the list; the winner should be chosen by side information, not by the decoder.

Chapter 7

2006:Jon Kleinberg

The central habit in this chapter is to state what local information an algorithm may use and what global question it must answer—and then to name the reinforcement rule whose equilibrium the answer represents.

7.1 An Opening Story: A Recipe Site Where Popularity Is Circular

Maya runs a small cooking site with four pages. Page A holds a famous lasagna recipe; page B, a weeknight-dinner blog, links to A; page C, a family recipe box, links to B and A; page D, a kitchen forum, links to C. Nobody links to D.

Which page answers “lasagna”? Count incoming links: A has two, B and C have one, D has none. Ask which is most *trusted* and the reasoning turns circular: A is trusted because good pages point to it, B because C points to it, C because D points to it, D because . . . nothing.

Popularity defined by popularity has no answer until we agree on a rule that breaks the circle.

7.2 The Problem

Network data arrives locally: a search engine sees millions of pages and the links among them; a person reaching a stranger sees only their own friends; a log shows one event at a time. Yet the questions worth asking are global: which pages are authoritative? Can someone reach a distant person using only nearby contacts? Is an hour of frantic activity a real story or ordinary noise?

A single local observation carries almost no information; the answer lives in the whole pattern.

The difficulty is *aggregation*. Three problems make this concrete:

- **Ranking.** Given the web's links, rank pages so authoritative ones rise to the top.
- **Navigation.** Given a network with long-range shortcuts, reach a distant target using only local information.
- **Burst detection.** Given a stream of timestamped events, decide when the rate genuinely rises and where bursts begin and end.

Each is easy to state and easy to get wrong: ranking is gameable, navigation can be blocked by information never seen, and bursts hide inside random variation.

7.3 The World Before

Early search engines matched query words against page text. Simple—and a standing invitation to cheat: a page repeating “lasagna recipe” twenty times outranked a better page using the

phrase once. Keyword-stuffed pages became the norm; raw text was trivial to game.

Counting incoming links was the natural fix: more pages pointing at you meant more people found you useful. Harder to fake with one page, but not with a thousand—a *link farm* is a cluster of pages built only to point at one target. Counting also hid the circularity of the recipe puzzle: a page is popular partly because *popular* pages link to it.

Social networks had a folklore theorem: Milgram’s letter experiments suggested a small diameter, about six handoffs between any two people, so navigation must be easy. But a person can only hand a letter to someone they know. Small diameter says a short path *exists*; it says nothing about whether someone who sees only their own acquaintances can *find* it. The inference—short paths imply easy navigation—is not merely unproven; it is false.

Bursts had a statistical description, but no algorithm for finding where a rate changes; eleven messages in ten minutes could be a burst or ordinary luck. Two ideas were missing: an *explicit model of mutual reinforcement*, turning circular importance into a computable fixed point, and an *explicit model of the information a local navigator has*, separating “a short path exists” from “a local rule can find it.” Kleinberg’s 2006 Nevanlinna Prize recognized precise answers to all three.

7.4 The New Idea

7.4.1 HITS: two roles, one fixed point

An *authority* is worth reading; a *hub* is worth following. They reinforce each other: good authorities are linked by good hubs, and good hubs link to good authorities.

Break the circle by iteration. Start every score at 1, then alternate:

- **Authority update:** new authority = sum of the hub scores of the pages linking to it.
- **Hub update:** new hub = sum of the authority scores of the pages it links to.

Rescale after each round. The mutual definition becomes concrete arithmetic that settles down.

In matrix language,

$$a \leftarrow A^T h, \quad h \leftarrow Aa,$$

and the stable scores are the principal singular vectors of the adjacency matrix A .

7.4.2 The small-world model: what a local navigator can use

Lay the network on a two-dimensional grid. Each person knows their grid neighbors and a few *shortcuts*—long-range contacts. The navigator's rule is greedy and myopic: move to the contact closest to the target; no one sees the map.

Suppose node u links to node v with probability proportional to $d(u, v)^{-r}$. The answer is sharp:

- $r = 2$: polylog expected steps;
- $r < 2$: too-uniform shortcuts—polynomial time;

- $r > 2$: too-local shortcuts—polynomial time.

The 2 is the grid's dimension. From distance δ to the target, the useful jumps land in the *halving ball*—nodes within $\delta/2$ of the target—about δ^2 of them, each reached with probability about $1/\delta^2$. A useful jump has comparable probability at every scale, so $\log_2 n$ halvings reach the target.

7.4.3 Bursts: separating a baseline from a self-exciting process

A burst is a stretch where one event makes the next more likely—*self-excitation*. The model runs two clocks: a normal state at rate λ_0 and a burst state at rate $\lambda_1 > \lambda_0$. Assigning each event to a state, best explaining the observed waiting times with a penalty per switch, turns a burst into an *inferred hidden structure* rather than a statistic.

7.4.4 The conceptual shift

All three models make explicit what information the algorithm may use and what rule converts it into an answer. The conclusions are precise and surprising: *local connectivity can generate useful global signals, but only when the definition of relevance is explicit; and small diameter does not imply navigability.*

7.5 How It Works: Worked Examples

7.5.1 Example 1: HITS by hand on four pages

Maya's site as a directed graph: $B \rightarrow A, C \rightarrow A, C \rightarrow B, D \rightarrow C$. Start all scores at 1.

Round 1, authority (a_j = sum of hub scores of pages linking to j):

$$a_A = h_B + h_C = 2, \quad a_B = h_C = 1, \quad a_C = h_D = 1, \quad a_D = 0.$$

Round 1, hub (h_i = sum of authority scores of pages i links to):

$$h_A = 0, \quad h_B = a_A = 2, \quad h_C = a_A + a_B = 3, \quad h_D = a_C = 1.$$

Round 2, authority:

$$a_A = h_B + h_C = 2 + 3 = 5, \quad a_B = h_C = 3, \quad a_C = h_D = 1, \quad a_D = 0.$$

Round 2, hub:

$$h_A = 0, \quad h_B = a_A = 5, \quad h_C = a_A + a_B = 5 + 3 = 8, \quad h_D = a_C = 1.$$

Collect the two rounds: authority $(2, 1, 1, 0) \rightarrow (5, 3, 1, 0)$, hub $(0, 2, 3, 1) \rightarrow (0, 5, 8, 1)$. A rises as the dominant authority, C as the dominant hub. Renormalizing after each round—dividing by the length of each score vector—keeps the iteration convergent without changing which page is largest. These scores are the *equilibrium of this rule*, not an objective fact about the pages.

7.5.2 Example 2: greedy routing on a 4 by 4 grid

Draw a 4×4 grid, target at $(4, 4)$, navigator at $(1, 1)$; without shortcuts the trip takes 6 steps.

Give $(1, 1)$ a shortcut to $(3, 3)$. Greedy compares options: $(1, 2)$ is 5 from the target, $(2, 1)$ is 5, the shortcut endpoint is 2. It jumps, then $(3, 3) \rightarrow (3, 4) \rightarrow (4, 4)$: three steps, and the jump cut the remaining distance from 6 to 2—roughly in half.

How should shortcuts be distributed? From the corner $(1,1)$ the grid has two nodes at distance 1, three at 2, four at 3, three at 4, two at 5, one at 6. Under $r = 2$ (probability proportional to $1/d^2$):

distance d	1	2	3	4	5	6
chance	57%	21%	13%	5%	2%	0.8%

Short jumps dominate; under a uniform law ($r = 0$) every distance is equally likely at about 6.7%, so no scale can be counted on. The contrast is decisive at scale n : the halving ball holds about δ^2 nodes, each reached with probability about $1/\delta^2$, so a useful jump has comparable chance at every scale and $\log_2 n$ halvings reach the target. Under $r < 2$ the longest links dominate and the halving chance collapses; under $r > 2$ shortcuts almost never reach half the gap. Both force polynomial time.

7.6 The Mathematics in Brief

HITS. The iteration

$$a \leftarrow A^T h, \quad h \leftarrow Aa$$

with normalization is power iteration for the symmetric matrices $A^T A$ and AA^T , converging to the principal singular-vector pair of A . *Why:* substituting gives $a \leftarrow A^T Aa$, and repeated multiplication by a symmetric matrix amplifies the largest-eigenvalue direction while shrinking all others.

Small-world navigation. Shortcuts follow

$$\mathbb{P}(u \text{ links to } v) \propto d(u, v)^{-r}.$$

On an $n \times n$ grid, greedy routing takes $O((\log n)^2)$ expected steps when $r = 2$ and $\Omega(n^\beta)$ for a constant $\beta > 0$ when $r \neq 2$. *Why:* under $r = 2$ the normalization sum is $\Theta(\log n)$, so the halving ball is hit with probability $\Theta(1/\log n)$ per shortcut; under $r < 2$ the halving chance is $\Theta((\delta/n)^{2-r})$; under $r > 2$ shortcuts never reach the ball. The exponent must equal the dimension.

Bursts. Waiting times are exponential at rate λ_0 in the normal state and $\lambda_1 > \lambda_0$ in the burst state; the model picks the assignment of events to states that best explains the observed waits, minus a penalty per switch. *Why:* a constant-rate model predicts long runs of very short waits to be exponentially unlikely, so when they occur a state switch is the parsimonious explanation; the penalty stops a new burst being declared around every event.

The distinction. A centralized algorithm knows the whole graph; a navigator sees only its position and contacts. “There is a short path” and “a local rule can find it” are different statements, and the small-world theorem draws the line between them.

7.7 Limits and Modern Views

The HITS equilibrium is not a neutral measure. A tightly connected community can dominate without being globally authoritative, and a feedback loop can lock in an early accident: top-ranked pages attract attention, attention creates links, links raise the rank [26]. A link is

not a causal statement: two pages may link because they share an audience, a platform may recommend both, and friendship and purchase may share a hidden cause. Rankings amplify these hidden causes, so modern work asks causal questions alongside graph questions [27].

Models are static while networks move, so a static eigenvector lags the system it summarizes; popularity attracts links partly because it is already popularity. Strategic actors game whatever signal is scored—link farms, coordinated bots, engineered cascades—so the evidence is adversarial [28]. Rankings encode past inequalities, attention measurements touch privacy, and an engagement-optimal ranking can worsen the decision it supports; fairness-aware and privacy-preserving models now share the agenda.

Kleinberg’s style survives these harder questions: name the mechanism, derive a testable prediction, then check which parts survive contact with data. The model is a hypothesis to test, not reality itself.

7.8 Exercises and Self-Study Guide

E1. Run one more HITS round on the worked example from $a = (5, 3, 1, 0)$, $h = (0, 5, 8, 1)$.

Confirm A stays the top authority and C the top hub.

Hint: Authority of a page is the sum of the hub scores of pages linking to it; hub is the sum of the authority scores of the pages it links to.

E2. On the three-page graph $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$, compute two HITS rounds from all-ones scores.

Which page becomes the top authority, and which the top hub?

Hint: $a_A = 0$ because nothing links to A ; C ’s authority and A ’s hub score grow fastest.

- E3.** On a 3×3 grid, place a shortcut from $(2, 2)$ to $(1, 1)$ and run greedy routing to target $(1, 1)$ from $(2, 2)$ and from $(1, 3)$. When does the shortcut help, and when is it invisible?

Hint: Greedy moves to the contact closest to the target; a shortcut is usable only from its source node.

- E4.** Two streams of twenty events each: S has one event every 100 minutes; T has nineteen events in the first 90 minutes, then silence. Same average rate. Which is a burst, and how does a model detect it?

Hint: Compare inter-arrival times: S has waits near 100 minutes, T has many tiny waits followed by one huge one.

- E5.** Describe a confounding variable that could make friendship links look as though they cause purchases.

Hint: A shared experience—a new TV show, a holiday, a company policy—can create both the friendship and the purchase.

- E6.** Remove the link $D \rightarrow C$ and rerun HITS for two rounds. Does A remain the top authority? Propose a simple stability test for a ranking when links change.

Hint: Rerun with the link deleted and compare the ordering of the top pages before and after.

- E7.** On the 4×4 grid, put a shortcut from $(1, 2)$ to $(4, 4)$ and route from $(1, 1)$ to $(4, 4)$. With full knowledge the shortest path is $(1, 1) \rightarrow (1, 2) \rightarrow (4, 4)$: two steps. Simulate greedy, breaking ties toward the larger row number. How many steps, and why does it miss the shortcut?

Hint: Greedy sees a shortcut only when standing at its source; the tie-break at $(1, 1)$ decides whether the walk ever reaches $(1, 2)$.

A final exercise in judgment: whenever a ranking is published or a path is followed, name

the reinforcement rule that produced the score and the local information the navigator was allowed to see. A number that cannot be traced to a stated rule is not a finding; it is a guess wearing notation.

Chapter 8

2010:Daniel Spielman

The central habit in this chapter is to translate a network into a matrix and to demand of every approximation one thing: it must preserve the energy geometry of the original graph.

8.1 An Opening Story: Springs on a Water Line

Six buildings stand in a row, each pair of neighbours joined by a pipe. Pipes resist differences: neighbours with pressures u and v stretch a pipe by $(u - v)^2$. For pressures 2, 0, -1 , the pipes stretch by $(2 - 0)^2 = 4$ and $(0 - (-1))^2 = 1$, total tension 5: every assignment of pressures carries one number, the network's total tension.

Two questions seem far apart. *Where is the weak joint?* A strong cluster meeting another across one rusty pipe lets pressures change on either side while tension stays small—the network is nearly two pieces. *What pressures does the network settle at?* One linked equation per building; for a city, a huge system. Daniel Spielman's 2010 Rolf Nevanlinna Prize recognized that both are the same information read two ways, and that a matrix is the bookkeeping making the

connection visible.

8.2 The Problem

Three problems about big networks, all questions a computer must answer quickly.

Solving huge sparse systems. Physical simulations produce one equation per point, coupling each point to its neighbours. A million-point mesh gives $Lx = b$ with a million unknowns and a handful of nonzeros per row; solving it directly is far too slow.

Finding weak cuts. Divide a network into two parts so little flows between them: communities, power-grid islands, circuit halves. The question is global—a weak cut is felt across the network, not at one edge—so examining edges one at a time answers nothing.

Explaining practical speed. The simplex method is excellent in practice yet has an astronomically slow worst case. Both are true; nobody could explain how.

Each was stuck because nobody could read a network's global behaviour, and because worst-case analysis ignored the noise in real inputs.

8.3 The World Before

Gaussian elimination costs about $n^3/3$ operations and destroys sparsity: a 1000×1000 grid yields a matrix with a million rows and about five million nonzeros, which elimination fills to roughly 10^{12} entries. So huge sparse systems were attacked with iterative methods—conjugate gradients, multigrid—which keep the matrix sparse and, in practice, converged quickly. Provable guarantees were far weaker than observed behaviour, and the fastest methods needed special

geometry.

Graph partitioning used flows and heuristics. Finding a minimum cut by max-flow is correct but expensive, and heuristic searches had no guarantee. No one had a theorem saying: this linear-system solution cuts the graph into provably good pieces.

Linear programming was the most embarrassing case. Klee and Minty built a small cube on which the simplex method visits an exponential number of corners—a formal disaster—yet practitioners ran it daily and loved it. Worst-case analysis had no vocabulary for the difference.

Two ingredients were missing: a translation of a discrete graph into a matrix recording its bottlenecks, currents, and mixing faithfully; and an analysis that respects reality—real inputs carry noise, so a proof should be able to call a pathological input fragile.

8.4 The New Idea

Stop looking at one edge at a time. Write the network as a matrix, define an energy on assignments, and let the matrix’s eigenvalues—numbers attached to the whole network—carry the answers.

8.4.1 The Laplacian: springs as bookkeeping

Build L with off-diagonal entry $-w_{ij}$ for the edge joining i and j and diagonal entry the sum of the weights meeting i : $L = D - A$, degree matrix minus adjacency, the *graph Laplacian*. For an assignment x ,

$$x^\top Lx = \sum_{(i,j)} w_{ij}(x_i - x_j)^2,$$

the opening story’s tension: the matrix turns every assignment into an energy.

Which assignments carry little energy for their size? Two dense parts joined by a thin thread admit $+1$ on one part, -1 on the other, differing only across the thread. The Rayleigh quotient $x^\top Lx / x^\top x$ measures this ratio; its minimum is the smallest eigenvalue, always 0 (the constant assignment). The *second-smallest* eigenvalue λ_2 is small exactly when the graph can be cut cheaply, and its eigenvector rounds into a provably good partition. Cheeger-type inequalities quantify this: the graph’s expansion h is sandwiched between a constant multiple of λ_2 and roughly its square root.

8.4.2 Spectral sparsification: a small graph with the same energy

A spectral sparsifier replaces the graph by a weighted graph with far fewer edges preserving every quadratic form within $(1 \pm \varepsilon)$:

$$(1 - \varepsilon)x^\top Lx \leq x^\top \tilde{L}x \leq (1 + \varepsilon)x^\top Lx \quad \text{for all } x.$$

“For all x ” is the honesty condition: the small graph imitates the large one on every assignment, so every computation on it is valid for the large one.

8.4.3 Recursive preconditioning: nearly-linear solvers

The energy view also speeds up solving $Lx = b$. An iterative solver converges in steps controlled by the spread of eigenvalues, and a *preconditioner*—a change of coordinates balancing scales—shrinks that spread. Spielman and Teng built it from the graph: a hierarchy of sparsifiers, each level sparse and guaranteeing the next. Result: Laplacian systems solved in nearly linear time,

order $m \log^{O(1)} n$.

8.4.4 Smoothed analysis: the fragility of worst cases

The fourth idea concerns the simplex method. Let an adversary choose the input, then add tiny random noise to every number. Degenerate configurations—corners where many constraints meet exactly—are destroyed, and Spielman and Teng proved the expected running time after the perturbation is polynomial. Worst cases still exist; they are fragile, and real inputs are never exactly them.

The shift is two-sided: a discrete problem can be translated into linear algebra without losing the structure that matters—bottlenecks become small eigenvalues, cuts become eigenvectors, flows become voltages—and worst cases can be fragile.

8.5 How It Works: Worked Examples

8.5.1 A path of three springs

Three vertices 1, 2, 3 in a line, every edge of weight 1:

$$L = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}.$$

For $x = (1, 0, -1)$, the edges stretch by $(1 - 0)^2 = 1$ and $(0 - (-1))^2 = 1$, so $x^T L x = 2$.

Multiplying by L gives $L(1, 1, 1) = (0, 0, 0)$, $L(1, 0, -1) = (1, 0, -1)$, $L(1, -2, 1) = 3(1, -2, 1)$, so

the eigenvalues are 0, 1, 3. The small eigenvalue 1 says the path splits for the price of one edge;

its eigenvector $(1, 0, -1)$ points at the cut $\{1\}$ versus $\{2, 3\}$.

8.5.2 Two tight clusters, one weak thread

Two tight pairs, 1, 2 and 3, 4, each joined by weight 10, plus a weak thread of weight 0.1 joining 2 to 3. With $x = (1, 1, -1, -1)$, the internal springs stretch nothing and only the thread stretches:

$$x^\top Lx = 0.1 (1 - (-1))^2 = 0.4,$$

while $x^\top x = 4$, so the Rayleigh quotient is $0.4/4 = 0.1$: a vector of size 4 carrying only 0.4 of energy—the graph splits almost for free. A degree count misses this; the energy reads the weak thread directly. The true second eigenvalue is about 0.0995, from $(1.01, 1, -1, -1.01)$.

8.6 The Mathematics in Brief

The Laplacian and its energy. For weighted adjacency w_{ij} and degree matrix D ,

$$L = D - A, \quad x^\top Lx = \sum_{(i,j)} w_{ij} (x_i - x_j)^2.$$

Why it is true: expanding the square, the diagonal degrees collect the $w_{ij}x_i^2$ terms and the off-diagonal entries split the cross terms.

The second eigenvalue detects cheap cuts. If h is the graph's expansion—the minimum, over sets S of size at most $n/2$, of the weight leaving S divided by $|S|$ —then

$$\frac{\lambda_2}{2} \lesssim h \lesssim \sqrt{2\lambda_2 d_{\max}}.$$

Why it is true: an assignment constant on each side of a cheap cut has energy about the cut weight; rounding a small-quotient vector to ± 1 loses only the stated factor.

Spectral sparsification. $\tilde{G}$ is an ε -spectral sparsifier of G when

$$(1 - \varepsilon)x^\top Lx \leq x^\top \tilde{L}x \leq (1 + \varepsilon)x^\top Lx \quad \text{for all } x.$$

Why it is true (sketch): sampling each edge with probability proportional to its conductance times its effective resistance matches expected energies on every vector; concentration extends the match to all vectors.

Nearly-linear Laplacian solvers. An m -edge graph's system $Lx = b$ can be solved, to precision ε , in time of order $m \log^{O(1)} n$. Why it is true: sparsify, sparsify the sparsifier, and repeat; each level preconditioners the next, and every level is sparse, so the iterative work is nearly linear.

Effective resistance and spanning trees. Sending one unit of current from a to b , voltages solve $Lx = \delta_a - \delta_b$, and the flow's energy is the effective resistance R_{ab} . In a weighted random spanning tree, edge e appears with probability

$$\Pr[e \in T] = w_e R_e,$$

its conductance times its effective resistance. Why it is true: a consequence of Kirchhoff's matrix-tree theorem, which counts spanning trees through the Laplacian's eigenvalues.

Smoothed analysis. An adversary chooses the input, then each entry is perturbed by independent noise of scale σ :

$$\mathbb{E}[\text{running time}] = \text{poly}(n, \text{input size}, 1/\sigma).$$

Why it is true: surviving requires the noise to preserve an exact degeneracy, which has negligible probability; away from degeneracies the work is bounded uniformly.

8.7 Limits and Modern Views

Nearly linear is not fast for small problems: the proofs carry large constants, and an elegant asymptotic solver can lose on a real machine to a well-tuned classical one. Finite precision matters: an eigenvector is never computed exactly, so a solver should be judged by its residual—how far Lx is from b —and a partitioning algorithm by the cut quality of the vector it actually produced. Dynamic graphs change, and a sparsifier for the current snapshot is stale after an update; keeping a hierarchy fresh in streaming or distributed settings remains hard. Spectra can hide local structure: two graphs can agree on their largest eigenvalues while differing locally, because an eigenvalue averages over the whole network.

The ideas travelled: spectral clustering treats communities as cheap cuts; image segmentation and scientific simulation gained provable matrix forms; network science reads bottlenecks as fragility; randomized linear algebra sparsifies matrices as a routine step. Spielman’s error-correcting codes share the instinct—a sparse bipartite graph connects variable nodes to check nodes, expansion ensures a small corrupted set meets many checks, and local updates catch corruption [29–31].

8.8 Exercises and Self-Study Guide

- E1.** Compute the Laplacian of the path $1 - 2 - 3 - 4$ with unit edges and the energy $x^\top Lx$ for $x = (1, -1, 1, -1)$ and $x = (0, 0, 1, 1)$. Which has the smaller Rayleigh quotient, and where does the path want to be cut?

Hint: For $x = (0, 0, 1, 1)$ only the middle edge stretches.

E2. In the two-cluster graph, show that $y = (2, 2, -1, -1)$ rounds to the same cut as $x = (1, 1, -1, -1)$

and has Rayleigh quotient 0.09. How far can pushing the sides apart take you?

Hint: Energy grows quadratically in the stretch, norm only linearly; the quotient can never fall below λ_2 .

E3. Explain why preserving every quadratic form is a stronger promise than preserving average degree.

Hint: A vector 1 on one cluster and -1 on the other has energy equal to the cut weight, which average degree says nothing about.

E4. Give a worst-case input that a small random perturbation would likely change, and explain why.

Hint: A linear program where three constraints meet at one corner: move any constraint a hair and a different pair binds.

E5. Relate expansion in a bipartite code graph to the ability of local checks to detect corruption.

Hint: If a small corrupted set touches few checks, errors hide there; expansion says every small set meets many checks.

E6. Derive the voltage solution for a path with one unit of current entering at one endpoint and leaving at the other.

Hint: One unit flows through every edge, so each edge carries voltage drop $1/w$; set the far endpoint to 0 and accumulate. Verify $Lx = b$ with $b = (1, 0, \dots, 0, -1)$.

E7. In the two-cluster graph, the weak thread has effective resistance 10. Use $\Pr[e \in T] = w_e R_e$ to show why every weighted random spanning tree must contain it.

Hint: For a bridge the probability is 1, so the identity forces $R_e = 1/w_e$.

A final exercise in judgment: whenever a small object claims to stand in for a large one, name the set of questions the small one answers honestly. If you can state the invariant—every

energy, within a factor—the substitution is mathematics; if you can only list examples that

happened to work, it is hope.

Chapter 9

2014:Subhash Khot

The central habit in this chapter is to judge a relaxation by its geometry: the angle between two vectors, not the list of labels, decides which approximation ratio is possible.

9.1 An Opening Story: A Party of Friends With Conflicting Labels

Four friends—Ana, Bo, Cece, Di—wear tags labeled 0 or 1. Each friendship carries a rule: some pairs must match, others differ. Ana–Bo match, Bo–Cece differ, Cece–Di match, Di–Ana differ.

Solution: Ana 0, Bo 0, Cece 1, Di 1.

Change one rule: Di–Ana must match. Walking around the circle forces Di’s number to equal Ana’s flipped, and to equal Ana’s—impossible; the best is three of four rules, 75%. A thousand friends and ten thousand rules behave the same at scale: each rule satisfiable alone, conflicts colliding around loops. Can a computer tell a party where nearly all rules hold from one where

only a tiny fraction do? That question earned Subhash Khot the 2014 Rolf Nevanlinna Prize.

9.2 The Problem

NP-hard problems have no efficient exact algorithm (unless $P = NP$), so we accept approximations within a fixed ratio of optimal. Two questions: *what ratio can an algorithm guarantee*, and *what ratio can be proved impossible*? A factor of two may be easy while 1.01 impossible—not for lack of cleverness but of proof: ruling out ratio r needs instances where the optimum is high, every cheap method is trapped far below it, and a too-good algorithm would solve an NP-hard problem.

Such proofs used to be bespoke; the field wanted one “mother” constraint system whose hardness transfers everywhere, and an explanation of why thresholds land exactly where algorithms sit.

9.3 The World Before

Exact optimization is NP-hard. Since Cook’s and Levin’s theorems, no efficient exact algorithm exists for essentially every interesting optimization problem; the field retreated to approximation and to proving matching impossibility.

Ratios, problem by problem. By the early 1990s each problem had its own folklore: scattered, ad hoc proofs producing constants like 2, 1.5, and 0.5, with no unifying explanation.

The PCP theorem. In the mid-1990s it let every NP proof be written so that a verifier checks a constant number of random bits; encoded as constraint systems, it became the first

general hardness engine. Håstad showed MAX-3SAT cannot be beaten past $7/8$, matching the trivial random assignment, but each problem needed a new construction and the tools stalled on two-variable problems like MAX-CUT.

An algorithm at 0.878. Goemans and Williamson (1995) approximated MAX-CUT within 0.878 via vectors and a random hyperplane. Unconditional PCP hardness reached only $16/17 \approx 0.941$; the matching $0.878 + \varepsilon$ was beyond the technology, and 0.878 had no advance explanation.

What was missing. A single conjectured base problem whose hardness transfers everywhere, and a language in which the transfer is automatic. Khot supplied both; the base problem is Unique Games.

9.4 The New Idea

9.4.1 A puzzle whose edges are permutations

A *unique game* is the party puzzle at scale. Vertices carry labels from $[k]$; each edge (u, v) carries a bijection π_e demanding $\pi_e(\text{label of } u) = \text{label of } v$. With two labels a permutation is just “same” or “flip.” Each constraint is one-to-one: fix one endpoint’s label and the other’s is forced. Every edge is locally trivial, so all difficulty is global, hidden in how permutations compose around cycles.

9.4.2 The promise that makes it hard

The Unique Games Conjecture (2002) concerns a *promise problem*: the instance is either nearly perfect—at least $1 - \delta$ of edges satisfiable—or almost empty—at most ε —for tiny $\delta, \varepsilon > 0$. The

conjecture: for every $\varepsilon, \delta > 0$ some label set size k makes *distinguishing the two cases NP-hard*.

Fully satisfiable instances are easy (guess one label and propagate); the difficulty is detecting global consistency when local demands never conflict.

9.4.3 Why the seed is so powerful

If problem P reduces *from* Unique Games, the promise survives: yes-instances become P -instances of value at least c , no-instances of value at most s . Any algorithm within ratio c/s would separate the cases. Because reductions are engineered outward from the game, c/s can be tuned to targets like $0.878 + \varepsilon$: one hard promise, many exact thresholds.

9.4.4 The geometry that fixes the threshold

The companion tool is the *semidefinite relaxation*: labels become unit vectors, constraints become angle statements. Rounding returns vectors to labels via a random hyperplane; an edge's survival probability depends only on the angle θ via θ/π . The *invariance principle*—high-dimensional discrete systems with no dominant variable behave like Gaussians—carries the game's discrete hardness into Gaussian geometry, to a threshold computed exactly, always matching the best-known algorithm.

The conceptual shift: *a single carefully designed constraint system can be the seed of hardness for many optimization problems; the geometry of the relaxation determines the threshold.*

9.5 How It Works: Worked Examples

9.5.1 Two four-person parties, label by label

Friends A, B, C, D ; labels $\{0, 1\}$; rules as equations $x_j = x_i \oplus b_{ij}$:

Edge	Rule	Equation
(A, B)	same	$x_B = x_A$
(B, C)	flip	$x_C = x_B \oplus 1$
(C, D)	same	$x_D = x_C$
(D, A)	flip	$x_A = x_D \oplus 1$

Set $x_A = 0$ and walk around: $x_B = 0$, $x_C = 1$, $x_D = 1$, and the last rule needs $x_A = x_D \oplus 1 = 0$ —

consistent. Two flips around the loop, and $0 \oplus 1 \oplus 1 = 0$; $(0, 0, 1, 1)$ satisfies all four rules.

Instance 2: change the last rule to “same,” $x_A = x_D$. The walk gives $x_D = x_A \oplus 1$, and the last rule demands $x_A = x_D = x_A \oplus 1$ —contradiction for either value of x_A . Each of the four assignments fails exactly one edge, so the best is $3/4 = 75\%$.

One flip instead of two. With two labels a cycle is fully satisfiable exactly when its flip count is even; with k labels: *the permutations around every cycle must compose to the identity*. Local constraints are individually satisfiable; only the product of permutations around closed walks decides global feasibility, and a label-recovery problem is hard precisely when this consistency is hard to detect.

9.5.2 MAX-CUT: vectors, then a random wall between them

MAX-CUT partitions a graph's vertices into two sides to maximize crossing edges—the two-label puzzle again, each edge demanding $x_u \neq x_v$.

The SDP relaxation assigns each vertex i a unit vector v_i and maximizes

$$\sum_{(i,j)} \frac{1 - v_i \cdot v_j}{2}.$$

Rounding: pick a uniformly random direction r ; put i on the side $\text{sign}(r \cdot v_i)$. Edge (i, j) is cut when the signs disagree, with probability depending only on $\theta = \arccos(v_i \cdot v_j)$:

$$\Pr[\text{cut}] = \frac{\theta}{\pi}.$$

Three checks: $v_i \cdot v_j = 0$ ($\theta = \pi/2$) gives $1/2$; $v_i = v_j$ ($\theta = 0$) gives 0 ; $v_i = -v_j$ ($\theta = \pi$) gives 1 .

At the worst angle, $\theta^* \approx 0.74\pi$ (about 133° , $\cos \theta^* \approx -0.69$): the SDP contribution is $(1 - (-0.69))/2 \approx 0.845$, but rounding cuts the edge with probability only 0.74 . The ratio $0.74/0.845 \approx 0.878$ is the minimum of $(\theta/\pi)/((1 - \cos \theta)/2)$ over all angles—exactly the Goemans–Williamson factor. Their theorem: for *every* graph, rounding any SDP solution cuts, in expectation, at least 0.878 times the SDP value, hence 0.878 times the optimum. Under Unique Games, beating $0.878 + \varepsilon$ is impossible (Khot, Kindler, Mossel, O'Donnell).

9.6 The Mathematics in Brief

The instance. A Unique Game is a graph (V, E) with label set $[k]$; each edge $e = (u, v)$ carries a bijection $\pi_e : [k] \rightarrow [k]$; an assignment a satisfies e when $\pi_e(a_u) = a_v$. Cycles are consistent exactly when their permutations compose to the identity.

The promise. Distinguish $(1 - \delta)$ -satisfiable (some assignment satisfies $\geq 1 - \delta$ of the edges) from ε -satisfiable (every assignment satisfies $\leq \varepsilon$). UGC: for every $\varepsilon, \delta > 0$ this is NP-hard for some label set size k .

The gap reduction. If a reduction from Unique Games to P preserves completeness c and soundness s , no algorithm approximates P within factor c/s , or it would separate the cases.

MAX-CUT. Relax to $\max \sum (1 - v_i \cdot v_j)/2$ over unit vectors; round by a random hyperplane with $\Pr[\text{cut}] = \theta/\pi$. The inequality $\theta/\pi \geq \alpha(1 - \cos \theta)/2$ with $\alpha \approx 0.878567$ yields the approximation factor, and UGC-hardness rules out $\alpha + \varepsilon$: the bounds meet.

The invariance principle. A low-degree multilinear polynomial of many weakly correlated discrete variables with no dominant variable is distributed nearly like the same polynomial on Gaussians with matching correlations—many small independent contributions converge to a Gaussian. It moves hardness from discrete games into Gaussian space, where isoperimetric theorems compute exact thresholds.

9.7 Limits and Modern Views

A conjecture, not a theorem. UGC remains open, so results conditional on it must be labeled as such; stronger forms were disproved, and the direct quantum analogue is false. The neighbouring *2-to-2 Games Conjecture* (constraints two-to-two instead of one-to-one) was proved in 2018 by Khot, Minzer, and Safra with Dinur, Kindler, and Safra, yielding new unconditional hardness for vertex cover (approaching $\sqrt{2}$, beating the long-standing 1.36) and independent set, and strengthening belief in UGC. Specialized families are understood; the general promise is not.

Unconditional gaps are weaker. An integrality gap shows one relaxation cannot certify a better ratio; it says nothing about stronger algorithms, while UGC hardness would settle the question.

Structure beats worst cases. Hardness is worst-case; sparse graphs, clustered data, and inputs with known distributions carry structure algorithms exploit. The right response to a barrier is diagnosis: ask what the fractional solution knows, what the integral solution requires, and which correlation the rounding step forgot.

Where the field went. Sum-of-squares hierarchies seek stronger relaxations; spectral and local algorithms exploit expansion; Raghavendra showed the best ratio for every constraint satisfaction problem is dictated by the geometry of its own relaxation [32,33].

9.8 Exercises and Self-Study Guide

- E1.** Verify both party instances by writing out all four assignments; then build a five-vertex cycle with exactly two flips and show it is fully satisfiable.

Hint: A two-label cycle is satisfiable exactly when its flip count is even; fix one label and propagate.

- E2.** Explain how the promise—“($1 - \delta$)- or ε -satisfiable”—differs from “is this instance fully satisfiable?”, and why the latter is easy.

Hint: A promise problem answers only on the two guaranteed input kinds; a fully satisfiable unique game is solved by propagating one guessed label.

- E3.** For vectors with $v_i \cdot v_j = 1/2$ and $v_i \cdot v_j = -1/2$, compute the angle θ , the SDP contribution $(1 - v_i \cdot v_j)/2$, and the cut probability θ/π . Which angle loses more?

Hint: $\arccos(1/2) = \pi/3$: cut probability $1/3$, contribution $1/4$, ratio $4/3$; only near $\theta^ \approx 0.74\pi$ does the ratio fall to 0.878.*

- E4.** A gap reduction has completeness $c = 0.9$ and soundness $s = 0.3$. If an algorithm returns at least OPT/r , which r separate yes from no by output size?

Hint: Separation needs $0.9/r > 0.3$, so $r < 3$; in general the factor ruled out is c/s .

- E5.** Identify a global property of the party cycle that a local pairwise relaxation cannot express.

Hint: Each flip is locally satisfiable by half of all label pairs; only the product of permutations around a loop records feasibility.

- E6.** Build a tiny integer program whose relaxation beats the integral optimum: cover the triangle,

$$x_A + x_B \geq 1, x_B + x_C \geq 1, x_A + x_C \geq 1, \text{ minimize the sum, } x_i \in \{0, 1\}.$$

Hint: $(1/2, 1/2, 1/2)$ costs 1.5; every integral cover costs 2.

- E7.** Give a rounding rule that preserves every marginal but violates a pairwise “must be equal” constraint with positive probability.

Hint: Independent rounding to 0 with probability p : marginals match, but two forced-equal variables agree only with probability $p^2 + (1-p)^2$.

A final exercise in judgment: whenever a relaxation seems to declare a problem easy, ask what the fractional solution knows, what the integral solution requires, and which correlation the rounding step forgot.

Chapter 10

2018:Constantinos Daskalakis

The central habit in this chapter is to keep existence and computation apart: a theorem that a solution exists is not an algorithm for finding it.

10.1 An Opening Story: A Price War Between Two Sellers

Two coffee sellers, Priya and Malik, face each other across the same street. Each morning each posts a price for a latte, and each profit depends on the other's price: undercut by a dollar and the commuters drift to your window. So each seller's *best* price is a moving target—whatever beats the other's price of yesterday. Starting from \$5 and \$5, Priya drops to \$4.50, Malik answers at \$5.25, Priya drops to \$4.25. They may settle on a stable pair, bounce forever, or wander.

A theorem guarantees a stable pair of prices exists: at some pair, neither seller, facing the other's price, would change anything. But the theorem does not say how to find it, nor what a month of daily adjustments will do. *A stable pair exists* and *here is how to find it* are different claims. This chapter is about the difference.

10.2 The Problem

In a game, each player's best choice depends on everyone else's. A *Nash equilibrium* is a collection of choices—possibly random, since a player may prefer to mix—such that nobody can improve by changing alone. Nash proved in 1950 that every finite game has an equilibrium. That settled existence. The problem here is *computation*: given the rules of a game, find an equilibrium. Existence and calculation are different questions, and a well-defined solution need not be algorithmically accessible.

10.3 The World Before

Nash's proof was non-constructive. It invokes Brouwer's fixed-point theorem: the map sending every profile of mixed strategies to a profile of best responses leaves an equilibrium unchanged—an equilibrium is a fixed point. Brouwer guarantees such a point sits in the compact region of profiles but gives no recipe for locating it.

Economists treated equilibria as predictions. The equilibrium of a market—prices where supply meets demand—was a forecast, on the implicit assumption that players or markets would reach it. For decades nobody asked whether anyone or any algorithm could actually find one.

Pivoting algorithms worked in practice. The Lemke–Howson algorithm and its relatives find equilibria of two-player games by walking around the polytope of profiles. They solve small games quickly, but carry no general guarantee: some games force exponentially long walks.

Continuous space, discrete search. Brouwer's theorem lives in a continuous region; a

computer searches a finite structure, and a grid does not help: a fixed point can sit between grid points, and the grid has exponentially many cells.

What was missing was a language for the difficulty of problems whose solutions are *guaranteed* to exist. Classes like NP capture “hard to decide”; for search problems with a promised answer, no comparable class existed, nor the reduction machinery to prove one such problem as hard as any other. Those tools were what Daskalakis and his collaborators supplied.

10.4 The New Idea

10.4.1 The path picture: PPAD

Consider a directed graph where every node has at most one arrow in and at most one arrow out. Such finite graphs split into paths and cycles. A node with no arrow entering is a *source*; one with no arrow leaving is a *sink*. Handed a source, following arrows must stop at a sink: the graph is finite, and a revisit would give some node a second incoming arrow—forbidden. The ends of paths come in pairs, and you were given one end.

Now make the graph *implicit*: 2^k nodes, not listed, with a small circuit reporting each node’s predecessor and successor. This is the setup of **PPAD** (Polynomial Parity Argument on Directed graphs): total search problems whose solution is guaranteed by exactly this parity/path argument. Given one endpoint of a path, find another—a node whose indegree differs from its outdegree. Following the path always works but may take 2^k steps.

10.4.2 Nash equilibrium is PPAD-complete

Daskalakis, Goldberg, and Papadimitriou showed that a game can encode the successor circuit of a PPAD problem: they built games with three or more players whose Nash equilibria encode the “other end of the path” of any PPAD instance, so finding an equilibrium is at least as hard as finding that endpoint. Chen and Deng extended this to two-player games; Chen, Deng, and Teng, even to *approximate* equilibria when the allowed error is small. This was the first natural problem known to combine guaranteed existence with provable hardness of search. The difficulty is not a missing algorithm; it is a structural property of the problem class.

10.4.3 The same framework, elsewhere

The reduction template spread to market equilibria, auction bidding, and mechanism design: a solution exists by a fixed-point or parity argument, but computing it is hard in general. Not a blanket claim of hopelessness—special cases have fast algorithms: two-player zero-sum games solve by linear programming; potential games settle under any improving best responses; few players, limited interaction, or special payoffs admit dedicated algorithms. *Easy structure versus hard general case is the whole point. The conceptual shift: a theorem that a solution exists is not an algorithm for finding it, and a problem can be guaranteed-solvable in principle yet provably intractable to search.*

10.5 How It Works: Worked Examples

10.5.1 Matching pennies: finding the mixed equilibrium

Row and Col each choose Heads (H) or Tails (T). Row receives $+1$ when the choices match and -1 when they differ; Col receives the opposite. This zero-sum game has no pure equilibrium: whichever pair you pick, one player wants to switch.

Let Row play H with probability p . Against Col's H , Row's expected payoff is $p(+1) + (1 - p)(-1) = 2p - 1$; against Col's T , it is $p(-1) + (1 - p)(+1) = 1 - 2p$. For Col to be willing to mix, Col must be indifferent between H and T , so

$$2p - 1 = 1 - 2p \implies 4p = 2 \implies p = \frac{1}{2},$$

and by symmetry Col also plays H with probability $\frac{1}{2}$.

The equilibrium: both randomize 50/50, each with expected payoff $2p - 1 = 0$. Check it: if Row switches to pure H while Col randomizes, Row's expected payoff is $\frac{1}{2}(+1) + \frac{1}{2}(-1) = 0$ —no gain. Nobody can improve by deviating, which is the definition of equilibrium. Note the *indifference principle*: at equilibrium, every action played with positive probability must give equal expected payoff, or the player would shift probability to the better one.

10.5.2 The PPAD path: existence you can feel but not always walk

Nodes $0, 1, 2, 3, 4$; edges $0 \rightarrow 1, 1 \rightarrow 2, 3 \rightarrow 4$. The graph is implicit: a successor circuit answers node by node,

$$\text{succ}(0) = 1, \text{succ}(1) = 2, \text{succ}(2) = \emptyset, \text{succ}(3) = 4, \text{succ}(4) = \emptyset,$$

with consistent predecessors $\text{pred}(1) = 0$, $\text{pred}(2) = 1$, $\text{pred}(4) = 3$. Sources (no incoming edge): 0 and 3; sinks (no outgoing edge): 2 and 4. Given source 0, follow: $\text{succ}(0) = 1$, $\text{succ}(1) = 2$, $\text{succ}(2) = \emptyset$. The walk stops at 2: the other end. Two sources, two sinks—ends of paths come in pairs, so knowing one end guarantees another exists.

Why hard? Here the whole graph is drawn, so the answer is obvious. In a real instance the graph has 2^k nodes presented only through the circuits: you evaluate them one node at a time, never seeing the whole graph, and the path from your source can have length 2^k . Walking it takes 2^k steps, though the guarantee that the other end exists is airtight. When the successor circuit encodes another hard problem, locating the other end is provably as hard as solving it.

10.6 The Mathematics in Brief

A *finite game* has n players with finite action sets A_i . A *mixed strategy* for player i is a distribution x_i over A_i ; a *profile* $x = (x_1, \dots, x_n)$ collects one per player, giving expected payoff $u_i(x)$. Strategy x_i is a *best response* when $u_i(x_i, x_{-i}) \geq u_i(y_i, x_{-i})$ for every y_i . A *Nash equilibrium* is a profile where everyone plays a best response—a fixed point of the joint best-response map. *Why existence is true*: profiles form a compact convex region, the map can be made continuous, and Brouwer’s theorem supplies a fixed point.

The *indifference principle* is the workhorse: at equilibrium, actions used with positive probability give equal expected payoff; worse actions get probability zero.

PPAD is the class of total search problems whose guaranteed solution follows from a path/parity argument on an implicitly defined graph: indegree and outdegree each ≤ 1 ; given a

source, find a node whose indegree differs from its outdegree. *Why the guarantee holds:* following the unique path from a source cannot loop, so it must reach a sink.

The completeness theorem. Computing a Nash equilibrium of a game with three or more players is PPAD-complete (Daskalakis–Goldberg–Papadimitriou), and remains so for two-player games and even for *approximate* equilibria (Chen–Deng; Chen–Deng–Teng). An ε -equilibrium is a profile where $u_i(x_i, x_{-i}) + \varepsilon \geq u_i(y_i, x_{-i})$ for all y_i : nobody gains more than ε by deviating. *Why:* games can encode successor circuits, so an equilibrium encodes the endpoint of a hidden path. *Consequence:* worst-case hardness coexists with fast algorithms on structured instances, and with learning dynamics that may converge to outcomes that are not equilibria.

10.7 Limits and Modern Views

PPAD-completeness is a *worst-case* statement, asymptotic in game size. It does not say real markets are unsolvable: practical instances usually carry structure—few players, symmetric payoffs, sparse interaction—that dedicated algorithms exploit.

Approximation does not dissolve the difficulty: an ε -equilibrium is hard when the required precision is part of the input, and the smaller the error, the more of the hard problem the game encodes. Discretization is no escape: replacing a continuous strategy space by a finite grid changes the game, and the discrete equilibria need not lie near the continuous ones. Learning dynamics may cycle or settle into stable non-equilibrium outcomes—for prediction, those may matter more than the equilibrium itself, shifting the question from *what is the equilibrium?* to *what outcomes can plausible processes generate?*

The field that grew from these results is *algorithmic economics*: auctions and mechanisms where truthfulness, feasibility, and computation fit together. Optimal transport supplies a geometric language for some mechanism-design problems, dual potentials playing the role of prices. The deep lesson: social concepts have computational structure—equilibrium, incentive compatibility, and optimal allocation are algorithmic tasks whose representations determine what can be found, approximated, or predicted [34, 35].

10.8 Exercises and Self-Study Guide

- E1.** Biased matching pennies: Row gains +3 when both play H , +1 when both play T , −1 on a mismatch; Col's payoffs are the negatives. If Row plays H with probability p and Col with probability q , find the equilibrium (p, q) and Row's expected payoff.

Hint: Col's mix q must make Row indifferent: equate $q(+3) + (1 - q)(-1)$ with $q(-1) + (1 - q)(+1)$; then use p to make Col indifferent.

- E2.** In ordinary matching pennies, show $(\text{both play } H)$ is an ε -equilibrium for $\varepsilon = 2$ but not an equilibrium, and that no pure profile is an equilibrium.

Hint: At (H, H) Row gets +1, Col −1; compute the best each could do by switching alone and take the larger gain.

- E3.** Explain why Nash's fixed-point existence proof gives no search algorithm.

Hint: Brouwer guarantees a point but no path to it; a grid search either misses it or faces exponentially many cells.

- E4.** Trace best-response dynamics in matching pennies from (H, H) and show the play cycles forever.

Hint: Row copies Col and Col plays the opposite: $(H, H) \rightarrow (H, T) \rightarrow (T, T) \rightarrow (T, H) \rightarrow (H, H)$.

- E5.** A seller's price is continuous in $[4, 7]$ with unique equilibrium \$5.37. Explain how forcing prices onto the grid $\{4, 5, 6, 7\}$ can change the equilibrium.

Hint: The grid game allows different deviations and output points; the continuous equilibrium may lie strictly between grid lines.

- E6.** No-regret learners in matching pennies never settle, yet their empirical frequencies converge to the equilibrium mix. Why is stable frequency compatible with unstable play?

Hint: Regret compares accumulated average payoff with the best fixed action; in a zero-sum game the frequencies encode the equilibrium while realized actions alternate.

- E7.** Pick one assumption a market or mechanism-design model relies on—quasi-linear utilities, a known type distribution, or fully rational agents—and explain what happens to the complexity analysis if it is removed.

Hint: Ask which part of the argument used it: a truthfulness proof needs quasi-linear utility, a hardness reduction needs structure encoding arbitrary instances—removing it reopens the complexity question.

A final exercise in judgment: whenever you are told that a solution exists, ask separately how a machine would find it, how the answer would be verified with a certificate, and whether the guarantee survives approximation. If you cannot name the certificate, the existence proof is not yet an algorithm.

Chapter 11

2022:Mark Braverman

The central habit in this chapter is to ask not only what a protocol communicates, but what uncertainty it must destroy before the communication can succeed.

11.1 An Opening Story: The Party That Never Happened

Two friends, Alice and Bob, sit in separate rooms planning a party. Each has privately decided whether they can come: a card reads “0” (no) or “1” (yes), and the party is worth planning only if *both* cards read 1. They must agree on the answer, but every text message costs one coin, and they want to spend as few coins as possible. The obvious plan costs two coins: Alice texts her card, Bob texts his. But watch what actually happens. If Alice’s card reads 0, the party is already off and Bob’s answer changes nothing—the second coin buys nothing at all. The smarter plan is for Alice to text first and for Bob to text only if her card reads 1. Then two coins are needed only when her card reads 1, which happens half the time; on the other half, one coin settles everything.

Now suppose the pack is rigged so that 0-cards are ninety-nine times more common than 1-cards. The second message almost never matters, yet a worst-case budget of two coins still stands. A planner who pays typical costs spends a bit over one coin; an accountant who charges for *information removed* rather than symbols sent demands only about a tenth of a coin. Three currencies, three prices for the same conversation. Deciding which price is honest, and learning to compute it, is the subject of this chapter—the work for which Mark Braverman received the 2022 Abacus Medal.

11.2 The Problem

Alice knows a private input x ; Bob knows a private input y . By exchanging messages according to an agreed protocol, they must compute a function $f(x, y)$ —for the party, “do both cards read 1?”—correctly on every pair of inputs. The classical question is easy to state: how many bits must they exchange in the worst case? The deeper question is harder: what is each bit doing?

A transcript is not just a string; it is a sequence of decisions made under partial knowledge. Alice’s message can reveal her card, rule out a whole class of worlds, coordinate a shared random experiment, or merely confirm an event Bob already believed. Counting symbols treats all four uses as equal, like pricing a house by counting its bricks. Two conversations of the same length

can have radically different value: one message splits two equally likely worlds, another confirms a world already 99% certain. The field's hardest open problems were about composition—must many independent copies cost many times one copy?—and about leakage: how much does a transcript reveal? Worst-case length could not answer either question.

So “how many bits?” and “how much does the conversation buy?” are different questions, and the party protocol is a microscope for the difference. The first message (Alice's card) always matters; the second (Bob's, sent half the time) matters only conditionally. A correct measure must charge them differently and let a long but nearly empty conversation cost almost nothing.

11.3 The World Before

Yao's 1979 model fixed the vocabulary: two players, private inputs, agreed rules, and a cost measured as the worst-case number of bits exchanged. The communication complexity $D(f)$ is the smallest worst-case length of any correct protocol, and the field built its power from lower-bound tools proving that specific functions genuinely need many bits.

Two tools dominated. A *fooling set* is a collection of input pairs that all demand separate transcripts: if (x, y) and (x', y') both map to output 1 but the crossed pairs (x, y') and (x', y) map to 0, then no two of them can share a conversation, so the protocol needs at least as many distinct transcripts as the set has elements. The *rank method* is more algebraic: write f as a matrix with rows x , columns y , entries $f(x, y)$. Every deterministic protocol carves the matrix into monochromatic rectangles, one per transcript, and each rectangle is rank-1, so the number of transcripts is at least the matrix's rank. For equality of two n -bit strings the matrix is the $2^n \times 2^n$ identity, of rank 2^n ; hence at least n bits are needed, and n suffice. These tools are sharp, elegant, and measure only length.

The tools shared a blind spot: they could not explain why a long protocol might be nearly free. The party protocol has worst-case length 2 bits, yet on a typical conversation its second message is empty and even on the long branch it says one obvious thing; there was no vocabulary for “this message was nearly predictable.” Worst-case counting also could not amortize: for deterministic protocols k copies trivially cost k times one copy, but the honest average cost per copy over a stream of independent copies, where randomized protocols may share randomness, had no sharp answer—and neither did the question of how much information a transcript leaks about the inputs. A field built to answer “how much must they say” had no language for “how much is revealed.”

The missing tool was a measure of communication that (i) decomposes round by round, (ii) adds over independent copies, and (iii) can be compared with transcript length. Information theory already had (i) and (ii) in entropy and the chain rule; the creative step was to point that machinery at interactive protocols and make the transcript itself the random variable being measured. The right unit of thought was not the symbol but the uncertainty destroyed.

11.4 The New Idea

The change of unit, in one sentence: charge a protocol for the information its transcript reveals about the inputs, not for the symbols it sends. A message the receiver could already predict is free; a message that splits the remaining uncertainty is expensive. This is the *internal information cost* of the protocol,

$$\text{IC}_\mu(\Pi) = I(X; \Pi \mid Y) + I(Y; \Pi \mid X),$$

where Π is the random transcript and the conditioning records what each party already knows: Alice holds X , Bob holds Y , and information already sitting on its own side of the border is not charged for crossing it. (With public randomness R shared by both, condition on R too, since

the shared coin is free.) The conditioning is not cosmetic: it is what makes the party protocol cost $3/2$ bits instead of 2, as the worked example will show.

Three consequences follow from this unit.

11.4.1 Compression: little information means few bits

If a protocol reveals almost nothing, its transcript should be reproducible with almost no communication. From Bob's point of view, Alice's message is a sample from a distribution she can compute; with shared randomness, both players can search the random tape for that sample together, paying only for the part Bob could not have predicted. The compression principle states that a transcript of internal information cost c can be simulated with communication proportional to c , up to overhead growing with rounds and error. Its sharpest form, Braverman–Rao's *information equals amortized communication*, is an equality: the honest per-copy cost of many independent copies of a function is exactly the internal information cost of one copy.

11.4.2 Direct sums: copies add up

If one copy needs information c , then k independent copies need kc . This direct-sum statement looks obvious and is the entire point: information, unlike worst-case length, satisfies the additivity amortized reasoning demands. Independent coordinates contribute independently to the transcript's information—the machinery is the chain rule. The subtlety is that conditioning on a whole transcript can couple coordinates that started independent; the fix is a random-coordinate embedding that converts the k -copy protocol into a one-copy protocol of the same per-copy cost.

11.4.3 Noise: checkpoints and entangled correction

Real channels flip bits, and one corrupted message changes the meaning of every later message: the parties may silently disagree about which history they share. Error correction cannot be bolted on beneath the computation, since the players may not even agree on what to correct. Robust interactive protocols must create checkpoints—frequent cheap confirmations that both parties stand on the same transcript—so computation and correction become entangled. Braverman showed that a protocol of length n can be simulated over a channel that flips bits at a fixed rate using only a constant factor more communication, with error as small as desired; the information viewpoint turned a believed polylogarithmic overhead into a constant.

11.5 How It Works: Worked Examples

11.5.1 A warm-up: how much does a noisy coin flip tell us?

Let X be a fair bit, so $H(X) = 1$. Suppose a friend tells us the value of Z , obtained by flipping X with probability p : if $X = 0$ then $Z = 1$ with probability p ; if $X = 1$ then $Z = 0$ with probability p . How much do we learn about X by hearing Z ?

Write $H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$ for the binary entropy function. Given X , the observation Z is a coin flip with bias p , so $H(Z | X) = H(p)$. Meanwhile Z is still a fair bit overall, because flipping a fair coin with probability p leaves it fair: $H(Z) = 1$. Therefore

$$I(X; Z) = H(Z) - H(Z | X) = 1 - H(p).$$

Check the endpoints. If $p = 0$ then $H(0) = 0$ and $I = 1$: Z always equals X , so hearing Z reveals X completely. If $p = 1/2$ then $H(1/2) = 1$ and $I = 0$: Z is a fresh fair coin flip, statistically independent of X . Now one interior value. For $p = 1/4$,

$$H(1/4) = -\frac{1}{4} \log_2 \frac{1}{4} - \frac{3}{4} \log_2 \frac{3}{4} = \frac{1}{2} + \frac{3}{4}(0.415) \approx 0.811,$$

using $\log_2 4 = 2$ and $\log_2(3/4) = \log_2 3 - 2 \approx -0.415$. So a $3/4$ -reliable observation is worth $I(X; Z) \approx 0.189$ bits—less than a fifth of a bit, because most of what it says was already predictable. That is the first concrete lesson of the chapter.

11.5.2 The two-bit AND protocol: a conversation worth $3/2$ bits

Return to the party. X and Y are independent fair bits, and the protocol is the smart one: Alice sends X ; if $X = 1$, Bob sends Y ; if $X = 0$, Bob sends nothing. The three possible transcripts and their probabilities are:

Transcript	Input	Probability
“0”	$X = 0$	$1/2$
“1,0”	$X = 1, Y = 0$	$1/4$
“1,1”	$X = 1, Y = 1$	$1/4$

Compute the internal information cost by hand, one term at a time.

Alice’s contribution. For $I(X; \Pi | Y)$, fix $Y = 0$. Then X is still a fair bit, so $H(X | Y = 0) = 1$, and the transcript determines X : “0” means $X = 0$, “1,0” means $X = 1$. Thus $H(X | \Pi, Y = 0) = 0$ and $I(X; \Pi | Y = 0) = 1$. The same with $Y = 1$ gives $I(X; \Pi | Y = 1) = 1$, so averaging over Y ,

$$I(X; \Pi | Y) = \frac{1}{2} \cdot 1 + \frac{1}{2} \cdot 1 = 1.$$

Bob’s contribution. For $I(Y; \Pi | X)$, split by X . If $X = 0$, the transcript is always “0” and says nothing about Y , contributing 0. If $X = 1$, the transcript is “1,Y” and determines Y exactly, contributing $H(Y | X = 1) - 0 = 1$. Averaging over X ,

$$I(Y; \Pi | X) = \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 1 = \frac{1}{2}.$$

The internal information cost is therefore $1 + \frac{1}{2} = \frac{3}{2}$ bits, even though the worst-case transcript carries two symbols. The missing half-bit is Bob’s message: worth a full bit when sent, it is sent only when $X = 1$, so on average it contributes half a bit. A worst-case accountant charges 2 bits; the information accountant charges $3/2$. As a check, the transcript distribution $1/2, 1/4, 1/4$ has entropy $\frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{4} \cdot 2 = \frac{3}{2}$ bits, so the external information cost $I(X, Y; \Pi)$ is also $3/2$ here.

Now the rigged pack: keep Y fair but let $\Pr[X = 0] = 1 - \delta$. Bob speaks with probability δ and then reveals Y fully, so $I(Y; \Pi | X) = \delta$. For either fixed Y , Alice’s message determines X , whose entropy is now $h(\delta)$ (the binary entropy of the biased bit), so $I(X; \Pi | Y) = h(\delta)$; the cost is $h(\delta) + \delta$. At $\delta = 1/2$ this reproduces $\frac{3}{2}$; at $\delta = 1/4$ it is about $0.811 + 0.25 \approx 1.06$ bits; as $\delta \rightarrow 0$ it tends to 0 while the worst-case transcript still holds two symbols. Information and worst-case length are different currencies indeed.

11.6 The Mathematics in Brief

The quantities are few, and each is a compact record of an idea already met. The model, precisely: a *deterministic protocol* for f is a finite binary tree whose internal nodes alternate between the two players; at a node on the player’s turn, a bit is sent whose value is a function of that player’s input and the transcript so far, and each leaf is labeled by the answer. The *communication complexity* $D(f)$ is the minimum, over correct protocols, of the worst-case depth. Every deterministic protocol of depth t partitions the input matrix into t monochromatic rectangles, one per transcript, and each rectangle is rank-1; the transcript count is therefore at least the rank of the matrix of f (the *rank method*), and at least the size of any fooling set. For a discrete random variable X with probabilities p_x , the entropy is the expected surprise,

$$H(X) = - \sum_x p_x \log_2 p_x,$$

measured in bits. The conditional entropy

$$H(X | Y) = \sum_y \mathbb{P}(Y = y) H(X | Y = y)$$

is the uncertainty about X remaining after hearing Y , averaged over what Y might say. The mutual information $I(X; Y) = H(X) - H(X | Y)$ is the reduction in that uncertainty, and it is symmetric. The chain rule

$$H(X, Y) = H(X) + H(Y | X)$$

is the algebraic reason everything decomposes: the entropy of a pair is the entropy of the first coordinate plus the conditional entropy of the second. Applied message by message to a transcript $\Pi = (M_1, \dots, M_r)$, it becomes

$$I(X; \Pi | Y) = \sum_{i=1}^r I(X; M_i | Y, M_{<i}),$$

charging each round exactly for the uncertainty that round removes, given everything already said.

The internal information cost of a protocol on input distribution μ is the quantity this chapter is about,

$$\text{IC}_\mu(\Pi) = I(X; \Pi | Y, R) + I(Y; \Pi | X, R),$$

with R the public randomness, free to both parties. The external information cost $I(X, Y; \Pi)$ instead measures what a third-party observer learns from the whole transcript—a question about leakage rather than negotiation. On top of these definitions stand two theorems. The *compression* theorem says that a transcript of internal information cost c can be simulated with communication roughly c plus an overhead depending on rounds and error—little information, few bits. The *direct-sum* theorem says that k independent copies under the product distribution satisfy

$$\text{IC}_{\mu^k}(f^k) = k \text{IC}_\mu(f) :$$

copies add exactly. Why? Independence makes the transcript's information decompose coordinate by coordinate through the chain rule; the caveat is that conditioning on a shared transcript can couple coordinates that started independent. The proof repairs this by embedding one copy into a random coordinate of the full protocol and averaging: a k -copy protocol, restricted to a randomly chosen coordinate, is a valid one-copy protocol of cost equal to the average per-coordinate cost. So one copy can never be cheaper than the average, and k copies need at least k times one copy. Information, unlike length, composes.

The sharpest form is an *equality*, Braverman–Rao's *information equals amortized communication*. Let $D_\mu^{\text{pub}}(f^{\otimes k})$ be the communication cost of the best public-coin protocol computing k independent copies of f under the product distribution. Then

$$\text{IC}_\mu(f) = \lim_{k \rightarrow \infty} \frac{D_\mu^{\text{pub}}(f^{\otimes k})}{k}.$$

The upper bound (amortized cost is at most information) is compression: a transcript of internal information cost c can be simulated with communication $\sim c$ plus overhead depending on rounds and error, so many copies can be handled at roughly their per-copy information. The lower bound is the direct sum above. Length has no such equality: worst-case length does not add under independent copies in any usable way, which is precisely why the field could not compute an honest per-copy price before information entered.

11.7 Limits and Modern Views

The framework is powerful and, like any measuring instrument, only as good as its calibration [1]. Information complexity is *distributional*: every value is computed under a chosen input distribution, and choosing it is a creative act. A distributional lower bound transfers to a worst-case statement only with care, since a protocol may be cheap on typical inputs and expensive on rare ones, and approximate protocols need explicit error accounting because allowing error changes which uncertainty must be destroyed. Multiparty protocols spread information over many correlated views; quantum protocols replace messages with states whose measurement can destroy information; adaptive settings generate transcripts whose future questions depend on past answers. Each setting needs a refined notion, but the invariant survives: a constrained process must transform uncertainty into knowledge, and that transformation should compose under the operation being studied.

The framework also grew outward [6]. In *streaming* algorithms, a small-space algorithm's answers about a long stream are a transcript in disguise, and communication lower bounds transfer to space through such embeddings. In *data structures*, one encodes a communication problem into memory and query patterns; a too-cheap data structure would yield an implausibly cheap protocol, an approach used for lower bounds on extended formulations of linear programs [5]. In *privacy*, the fields meet in the same object: differential privacy demands that transcript distributions under neighboring databases be hard to distinguish, and Pinsker's inequality

$$\|P - Q\|_{\text{TV}} \leq \sqrt{\frac{\ln 2}{2} D(P\|Q)}$$

(for base-2 divergence) quantifies the bridge from small divergence to small distinguishing power. The lesson is to match the quantity to the guarantee: mutual information is an average while privacy is a worst case over neighbors, so low information does not automatically buy worst-case privacy [4]. Information complexity did not replace communication complexity; it changed the unit of thought, and that unit now circulates wherever a distributed computation must account for what it reveals.

11.8 Exercises and Self-Study Guide

- E1.** In the noisy-bit model, compute $I(X; Z) = 1 - H(p)$ for $p = 1/3$ and $p = 1/8$, and verify the endpoints $p = 0$ and $p = 1/2$ again from scratch.
Hint: Use $H(1/3) \approx 0.918$ so $I \approx 0.082$, and $H(1/8) \approx 0.544$ so $I \approx 0.456$; at $p = 0$ the observation is exact, at $p = 1/2$ it is a fresh fair flip.
- E2.** Two correlated bits have joint probabilities $\mathbb{P}(0,0) = \mathbb{P}(1,1) = 3/8$ and $\mathbb{P}(0,1) = \mathbb{P}(1,0) = 1/8$. Compute the marginals, $H(X)$, $H(Y)$, $H(X,Y)$, and $I(X;Y)$ directly from the joint table.
Hint: Both marginals are fair, so $H(X) = H(Y) = 1$ and $H(X,Y) = 2 \cdot \frac{3}{8} \log_2 \frac{8}{3} + 2 \cdot \frac{1}{8} \log_2 8 \approx 1.811$; hence $I(X;Y) \approx 0.189$ bits—the same number as the $p = 1/4$ noisy bit, though the mechanism differs.
- E3.** Using the same joint table, verify the chain rule $H(X,Y) = H(X) + H(Y | X)$ by computing $H(Y | X = 0)$ and $H(Y | X = 1)$ and averaging.
Hint: Given $X = 0$, Y takes value 0 with probability $3/4$; both conditional entropies are $H(3/4) = H(1/4) \approx 0.811$, so $H(Y | X) \approx 0.811$ and the sum with $H(X) = 1$ matches $H(X,Y)$.
- E4.** Design a public-coin protocol for equality of n -bit strings with error at most $1/8$. State precisely which distributional assumption, if any, your information calculation would need.
Hint: Both parties use public randomness to pick a hash h from a 2-universal family into $\{0,1\}^3$ and compare $h(x)$ with $h(y)$; distinct strings collide with probability at most $1/8$. The correctness claim needs no input distribution, but any information-cost computation must fix one, e.g. uniform inputs.

E5. Take a one-round protocol in which Alice’s message comes from a distribution that depends on her private input—say message “1” only when her input is exceptional, probability δ —and attempt to compress it with shared randomness. Identify exactly where the simulation needs Alice’s input.
Hint: Shared randomness can sample from a fixed distribution, but Alice’s message distribution is conditional on her secret input. The simulator must know when to send the rare message, and describing that event costs about $\log_2(1/\delta)$ bits, which is the information the message really carries.

E6. Formulate a privacy-versus-computation question for a streaming algorithm: what plays the role of the transcript, and what would you charge?

Hint: Let the transcript be the sequence of query answers; charge for information about individual stream items. Adding noise to answers shrinks the divergence between transcripts of neighboring streams (Pinsker bounds the distinguishing power) while raising the error of the answers—the two forces meet in the transcript distribution.

E7. Redo the AND-protocol cost with $\Pr[X = 0] = 1 - \delta$ for $\delta = 1/4$, and check that $\delta = 1/2$ reproduces the $3/2$ -bit cost.

Hint: The answer is $h(\delta) + \delta \approx 0.811 + 0.25 \approx 1.06$ bits: Bob contributes δ because he speaks exactly when $X = 1$, and Alice contributes $h(\delta)$ because her message determines X for either fixed Y .

A final exercise in judgment: when a protocol surprises you by being long but information-cheap, name the messages that are nearly predictable and the uncertainty each one still removes. If you cannot say what a message buys, you cannot say what it costs.

Chapter 12

2026:Shayan Oveis Gharan

The central habit in this chapter is to encode a combinatorial family as a polynomial, and to read the polynomial's curvature, zeros, and spectra as information about the family.

12.1 An Opening Story: Which Trails Should Stay?

A small nature reserve has four campsites—north, east, south, and west—joined by five trails forming a square with a diagonal. The rangers must close some trails for maintenance while keeping every campsite reachable from every other. A minimal way to keep the reserve connected is to keep exactly three trails, with none wasted on loops. There are $\binom{5}{3} = 10$ ways to choose three of the five trails, but two of those choices form a closed triangle and strand the fourth campsite, leaving 8 workable choices. Each such choice is a *spanning tree*: three trails that touch all four campsites with no wasted loops.

Now suppose the reserve chooses among the 8 trees uniformly at random. How likely is each trail to survive? Listing all eight trees answers directly: the diagonal appears in 4 of them, so it survives with probability $4/8 = 1/2$; each side trail appears in 5, so it survives with probability $5/8$. That was easy because the reserve is tiny.

Now add campsites. The complete network on n campsites—every pair directly linked—has n^{n-2} spanning trees: 16 for $n = 4$, about 10^{61} (a number with 61 digits) for $n = 40$. The questions “how many spanning trees?” and “how likely is each trail?” still make sense, but no one will answer them by listing. This is where Oveis Gharan’s work begins: the discrete list hides a continuous object—a polynomial—that answers both questions with no listing at all. The 2026 Abacus Medal recognized this habit of reaching for a hidden geometric object when a combinatorial process looks too irregular to control [2].

12.2 The Problem

Three problems, all phrased as questions a computer must answer without writing out an astronomically large list.

Sampling from a huge family. Fix a family of combinatorial objects—the spanning trees of a graph, the bases of a matroid, the independent sets of a matroid—and attach a positive weight to every object (for example, the product of weights on its edges). Draw an object at random with probability proportional to its weight. For the four-campsite reserve this is a coin flip among eight trees; for a network of ten thousand sites the family is unimaginably large, yet the answer must come back in seconds.

Approximating the shortest tour. Given cities with distances obeying the triangle inequality (going directly between two cities is never longer than routing through a third), find the shortest tour that visits every city exactly once. The exact optimum is believed to be computationally

hopeless, so the target is a *ratio*: a tour guaranteed to be at most α times the optimum, with α as close to 1 as possible.

Proving counting inequalities. Count the independent sets of a matroid by size: I_k independent sets of size k . For the uniform matroid on 4 elements of rank 2, the counts are $I_0 = 1$, $I_1 = 4$, $I_2 = 6$. Are such sequences always *log-concave*, each term at least the geometric mean of its neighbors, $I_k^2 \geq I_{k-1}I_{k+1}$? Mason conjectured a much stronger normalized version in 1978; it stayed open for more than four decades.

All three problems are hard for the same reason: the family is too large to look at, so every argument must summarize the whole family with a smaller object that still remembers the answers.

12.3 The World Before

Before 1976, the travelling salesperson problem had a good heuristic reputation but no provable guarantee. Christofides changed that with an algorithm that is almost ridiculously short to describe, and whose $3/2$ approximation ratio then resisted improvement for roughly forty-five years [7].

Christofides' architecture. First build a minimum spanning tree: connect all cities with the cheapest possible set of roads. This costs at most the length of an optimal tour, because deleting one road from any tour leaves a spanning tree. Second, look at the vertices where the tree has an odd number of incident edges—they cannot close into a valid tour yet—and add a minimum-weight perfect matching among exactly those vertices. That matching costs at most half an optimal tour, because an optimal tour, restricted to the odd vertices, splits into two matchings of total length at most the tour. Third, *shortcut*: walk along the tree and the matching and, whenever the walk revisits a city, jump over it. The triangle inequality guarantees that shortcuts never lengthen the tour. Total: $1 + \frac{1}{2} = \frac{3}{2}$ times optimal.

Why $3/2$ survived. The proof is tight: there are instances where the tree alone costs a whole tour and the matching is forced to cost half a tour at the same time. Every improvement attempt used the same division of labor—one argument pays for connectivity, a separate argument pays for parity repair—and each layer kept paying the same bill. After decades it became clear that any real improvement must break the coupling: it must choose a spanning tree whose random choices are correlated with the parity repair that will follow. That requires a distribution over trees with very specific statistical properties, which is exactly what nobody had.

Markov chains, one family at a time. To sample from a huge family, a standard tool is a *Markov chain*: start with any object, make small random changes (drop an edge, add an edge), and hope the walk quickly forgets where it started. The obstacle is proving *rapid mixing*. Before the work in this chapter, mixing proofs were done family by family with ad hoc tools—conductance arguments that hunt for a bottleneck, couplings that pair two walks and show they meet. Each success looked different, and there was no common certificate a newcomer could reuse.

Log-concavity sat in convex geometry. Log-concave sequences and functions had a rich life in convex geometry and probability—Brunn–Minkowski theory, Alexandrov–Fenchel inequalities, the geometry of polytopes—but almost nobody connected them to algorithms. Mason's 1978 conjecture about the normalized counts $I_k / \binom{n}{k}$ sat exactly at this intersection: clearly combinatorial, provable by no combinatorial tool known then.

12.4 The New Idea

The unifying move is to change the type of the object being studied. A family of discrete objects becomes a single polynomial, and the polynomial's geometry—where it is concave, where its

zeros live, what its spectrum is—becomes a certificate that algorithms can consume. Four named ideas form the toolkit.

12.4.1 A family becomes a polynomial

For a graph G with an edge variable z_e per edge, define the *spanning-tree polynomial*

$$T_G(z) = \sum_{\mathcal{T} \text{ spanning tree}} \prod_{e \in \mathcal{T}} z_e,$$

one monomial per tree. The polynomial is a storage device: the coefficient of a monomial counts the trees containing that set of edges. More importantly, derivatives are exactly the operations an algorithm needs. Setting $z_e = 0$ *forbids* edge e ; differentiating with respect to z_e *forces* it. The ratio $z_e \partial_{z_e} \log T_G$, evaluated at $z_e = 1$, is the probability that a uniformly random spanning tree contains edge e . A combinatorial question about a huge family has become a calculus question about one object.

12.4.2 Log-concavity and negative dependence

A positive polynomial p is *log-concave* on the positive orthant when $\log p$ bends downward in every direction there. The word is geometry, but the content is statistics: log-concavity of a generating polynomial is equivalent to *negative dependence*—including one element makes the others less likely, in a controlled way. It also survives the two operations an algorithm needs: conditioning (take a derivative) and restriction (set variables to zero). A proof that a polynomial is log-concave is therefore a promise that the whole sequence of conditional distributions the algorithm creates will stay well behaved.

Stable polynomials are the mechanism that produces log-concavity. A multivariate polynomial is stable when it has no zeros with all variables in the upper half-plane; this is the natural generalization of a one-variable polynomial with all real roots. Real-rootedness was already known to force log-concavity of coefficient sequences, and stability inherits that power in many variables while being closed under exactly the operations—differentiation, specialization, limits—that a proof needs.

12.4.3 Spectral independence: a local certificate

For a distribution on subsets, define the *influence* of one element on another: how much does conditioning on one element being present change the marginal probability of another? A large influence entry is a suspicion of a bottleneck; a large *eigenvalue* of the influence matrix is the real disease, because a matrix can have small entries yet greatly amplify a well-chosen direction. *Spectral independence* requires the largest eigenvalue of every influence matrix—under *every* partial assignment the chain might encounter—to stay bounded. This converts “prove there is no hidden bottleneck” into “bound the eigenvalues of some small matrices,” and it is exactly the local certificate a fast-mixing proof needs.

12.4.4 Entropic rounding: how 3/2 fell

The last idea closes the TSP story. Solve a fractional relaxation of the tour problem: a point x in a polytope that mixes connectivity and parity information and costs at most the optimal tour. Then round it to a real tour. The old rounding rules chose edges independently and paid the same parity bill every time. The new rule is *entropic rounding*: choose the distribution over spanning trees with maximum entropy among all distributions whose marginals match x , then sample from it. Because the tree polynomial is log-concave, the sampled tree is globally correlated in exactly the way the parity repair needs, and the two bills—connectivity and parity—can no

longer both be maximally expensive at once. The result: approximation ratio $3/2 - \varepsilon$ with $\varepsilon = n^{-c}$ for some constant $c > 0$, the first improvement over Christofides in nearly half a century.

The conceptual shift, in one line: *a family of trees can be a determinant; a distribution can be a polynomial; correlations can be an operator; a mixing proof can be a curvature argument.*

12.5 How It Works: Worked Examples

12.5.1 The triangle: three trees, one polynomial

Take the triangle on vertices $\{1, 2, 3\}$ with edge weights a (between 1 and 2), b (between 2 and 3), and c (between 1 and 3). Every spanning tree omits exactly one edge, so the spanning-tree polynomial is

$$T(a, b, c) = ab + ac + bc.$$

Set $a = b = c = 1$. Then $T = 3$: the triangle has three spanning trees. Edge a appears in the trees $\{ab, ac\}$, two out of three, so a uniformly random tree contains edge a with probability $2/3$.

Now the polynomial answers the same question by calculus. The derivative

$$a \frac{\partial T}{\partial a} = a(b + c) = 1 \cdot (1 + 1) = 2$$

counts, with weight, the trees containing edge a ; dividing by $T = 3$ gives $2/3$. Equivalently, the *logarithmic derivative*

$$a \frac{\partial}{\partial a} \log T = \frac{a(b + c)}{ab + ac + bc} = \frac{2}{3}$$

is the marginal probability. *A derivative that looks analytic is a probability.*

12.5.2 A resistor agrees

Now treat each edge as a 1-ohm resistor and compute the *effective resistance* between the endpoints of edge a : the resistance of the whole network seen from those two terminals. The route through b and c in series has resistance $1 + 1 = 2$ ohms; the direct edge a is a 1-ohm path in parallel with it, so

$$R_{\text{eff}} = \frac{1 \cdot 2}{1 + 2} = \frac{2}{3} \text{ ohms.}$$

The electrical identity behind random spanning trees (Kirchhoff) says: the probability that an edge belongs to the random tree equals its conductance times the effective resistance between its endpoints. The conductance is 1, so the probability is $2/3$ again. Two computations that look completely different—counting trees, and reducing resistor networks—deliver the same number. That coincidence is the pattern: *write the partition function, differentiate it, read the derivative as a marginal, and recognize the marginal as a physical quantity.*

The identity is not a special case of equal weights. Try $a = 1$, $b = 2$, $c = 3$: the polynomial gives $T = 11$ and probability $a(b+c)/T = 5/11$. The series path b, c has resistance $1/2 + 1/3 = 5/6$; in parallel with the 1-ohm edge,

$$R_{\text{eff}} = \frac{1}{1 + 6/5} = \frac{5}{11},$$

and conductance times resistance is $5/11$ again.

12.5.3 Mason's conjecture on a small matroid

A sequence $x_0, x_1, \dots$ is *log-concave* when $x_k^2 \geq x_{k-1}x_{k+1}$ for every k : each term is at least the geometric mean of its neighbors. The binomial coefficients $\binom{4}{k} = 1, 4, 6, 4, 1$ are log-concave, since $4^2 = 16 \geq 1 \cdot 6 = 6$ and $6^2 = 36 \geq 4 \cdot 4 = 16$.

The uniform matroid $U_{2,4}$ keeps every subset of $\{1, 2, 3, 4\}$ of size at most 2 independent. Its counts are $I_0 = 1$, $I_1 = 4$, $I_2 = 6$ —the entries of the binomial row, by design. Mason’s conjecture (now a theorem of Anari, Liu, Oveis Gharan, and Vintant [10]) says the *normalized* counts are log-concave:

$$\left(\frac{I_k}{\binom{n}{k}}\right)^2 \geq \frac{I_{k-1}}{\binom{n}{k-1}} \cdot \frac{I_{k+1}}{\binom{n}{k+1}},$$

where n is the number of elements of the matroid—for $U_{2,4}$, $n = 4$. (Do not normalize by the rank $\binom{r}{k}$; the ground-set size n is the right denominator.) Clearing denominators gives the working form

$$I_k^2 \geq I_{k-1} I_{k+1} \frac{(k+1)(n-k+1)}{k(n-k)}.$$

Check $k = 1$ for $U_{2,4}$:

$$I_1^2 = 16, \quad I_0 I_2 \frac{2 \cdot 4}{1 \cdot 3} = 1 \cdot 6 \cdot \frac{8}{3} = 16.$$

Equality—the uniform matroid is the perfectly balanced case, exactly where the bound is sharp.

The conjecture is meaningful for matroids that are not uniform. Take the graphic matroid of the complete graph K_4 : its elements are the 6 edges, and independent sets are forests. The counts are $I_0 = 1$, $I_1 = 6$, $I_2 = 15$ (any two edges), $I_3 = 16$ (the spanning trees, by Cayley’s formula $n^{n-2} = 16$), and $I_4 = 0$. Here $n = 6$, and the $k = 2$ check reads

$$I_2^2 = 225, \quad I_1 I_3 \frac{(3)(5)}{(2)(4)} = 6 \cdot 16 \cdot \frac{15}{8} = 180,$$

and $225 \geq 180$ holds. The plain sequence 1, 6, 15, 16 is log-concave too ($15^2 = 225 \geq 6 \cdot 16 = 96$), but the normalized inequality is strictly stronger, and it is the strong form that powers the algorithms: it is exactly the statement that the independence polynomial is log-concave in the sense needed for fast sampling.

12.6 The Mathematics in Brief

The spanning-tree polynomial is a determinant. The matrix-tree theorem: form the weighted Laplacian L with diagonal entries equal to the sum of the edge weights incident to a vertex and off-diagonal entries $-w_{uv}$; delete any one row and column; the determinant is exactly $T_G(z)$. Determinants are more than a computation trick: they give the polynomial a spectral life, including its stability, that pure combinatorics cannot see.

Log-concavity is a statement about covariance. For a positive polynomial p , the Hessian

$$\nabla^2 \log p = \frac{p \nabla^2 p - (\nabla p)(\nabla p)^\top}{p^2}$$

is negative semidefinite exactly when p is log-concave. The first derivatives of $\log p$ are marginal probabilities; the second derivatives, after subtracting products of first derivatives, are conditional covariances. Negative semidefiniteness says these covariances are nonpositive in every direction: negative dependence, uniformly. The “why”: the numerator is a covariance-like object in disguise, and concavity forbids any direction in which including one element systematically amplifies another.

Influence matrices. For a distribution on binary variables and a partial assignment τ , define

$$\Psi_{ij}^\tau = \mathbb{P}(X_j = 1 \mid X_i = 1, \tau) - \mathbb{P}(X_j = 1 \mid X_i = 0, \tau).$$

Spectral independence requires $\lambda_{\max}(\Psi^\tau) \leq C$ for every τ . The eigenvalue, not the entries, is the right measure: the all-ones matrix has every entry 1 but operator norm m , and a chain that must navigate such a collective push needs to know about it.

Local certificates, global mixing. The payoff theorem (Asadi, Oveis Gharan, and Saberi [8]; sharpened by Anari, Liu, Oveis Gharan, and Vinzant [11]) says that log-concavity or spectral independence implies fast mixing of the natural local chain, with tight rates. Sampling then becomes a usable subroutine, and counting rides on it: a telescoping product of partition-function ratios, each factor estimated from samples at a nearby parameter, yields a fully polynomial randomized approximation scheme for counting matroid bases [9].

The TSP, in two layers. The subtour-elimination linear program relaxes tours to fractional vectors x_e on edges, costing at most the optimum tour. The rounding layer chooses a distribution over spanning trees whose marginals match x and which has maximum entropy among all such distributions; the log-concavity of T_G supplies the negative correlation that keeps the parity-repair bill in check. The layers are inseparable: a relaxation that sees structure is useless without a rounding rule that respects it.

12.7 Limits and Modern Views

The certificate-based picture has boundaries worth knowing before they bite.

Certificates can be hard to verify. A polynomial may be log-concave or stable, but checking that property computationally is not free, and a theorem about an inaccessible certificate does not by itself produce an algorithm.

Chain steps can be expensive. Fast mixing is measured in number of steps; the steps themselves must be cheap, and a local move over a huge family may involve a determinant or a linear-system solve per step. The true running time is steps times per-step cost.

Constants hide in guarantees. The metric TSP improvement is $3/2 - \varepsilon$ with $\varepsilon = n^{-c}$ [7]—a genuine theoretical milestone that does not change the number on a real logistics invoice. Approximation theory measures ratios asymptotically; practice measures constants [2].

Log-concavity fails at phase transitions. Real systems have regimes—high density, sharp thresholds—where the balanced, negatively dependent world breaks down and bottlenecks reappear. The certificates are true only where their hypotheses hold, and knowing where they fail is part of using them.

The field moved outward from these boundaries. The geometry-of-polynomials program now touches approximate counting, high-dimensional statistical physics, convex geometry, and algebraic combinatorics via Hodge theory; spectral independence has become a standard tool for spin systems and other Gibbs distributions; entropic rounding opened a research line on correlation-aware rounding beyond TSP. Open directions include weaker stability notions that still imply negative dependence, adaptive and nonreversible chain designs, and approximate or learned certificates. The habit of this chapter—encode, then read geometry—survives every one of these changes.

12.8 Exercises and Self-Study Guide

- E1.** List all spanning trees of a path on four vertices and write its weighted spanning-tree polynomial.

Hint: A spanning tree of a four-vertex graph has three edges, and the path has exactly three. There is exactly one tree; the polynomial is a single monomial, and every edge marginal is 1. Compare with the triangle, where every tree omits one edge.

- E2.** For the triangle with weights $a = 1$, $b = 2$, $c = 3$, compute the probability that a random weighted tree contains edge a , in two ways: from the polynomial $T = ab + ac + bc$, and from the series-parallel effective resistance between the endpoints.

Hint: Both give $5/11$: the series path b, c has resistance $1/2 + 1/3 = 5/6$, and in parallel with the 1-ohm edge the effective resistance is $1/(1 + 6/5) = 5/11$.

- E3.** Verify Mason's inequality $I_k^2 \geq I_{k-1}I_{k+1}(k+1)(n-k+1)/(k(n-k))$ for $U_{2,4}$ at $k=1$ and for the graphic matroid of K_4 at $k=2$.
Hint: Use $n=4$ for $U_{2,4}$ and $n=6$ for K_4 (the number of edges, not the rank 3). The counts are 1, 4, 6 and 1, 6, 15, 16; the two checks are equality and $225 \geq 180$.
- E4.** Build the two-state Markov chain with transition matrix $\begin{pmatrix} 1-a & a \\ b & 1-b \end{pmatrix}$. Find its stationary distribution and eigenvalues, and decide when mixing is fast.
Hint: The stationary distribution is proportional to (b, a) ; the eigenvalues are 1 and $1-a-b$, so the spectral gap $a+b$ controls the mixing speed.
- E5.** Give a matrix whose every entry is small but whose operator norm is large.
Hint: The all-ones matrix J has $\|J\| = m$ while every entry is 1: the vector $(1, \dots, 1)$ is an eigenvector with eigenvalue m . Small entries can still push collectively.
- E6.** Design a local Markov chain for sampling independent sets of a small graph, and identify a likely bottleneck.
Hint: Use Glauber dynamics: add or delete a vertex while maintaining independence. On a star graph the chain must cross between "center chosen" and "center absent"; thin cuts between such regions control the mixing time.
- E7.** Compute the total variation distance between the distributions $\mu = (0.9, 0.1)$ and $\nu = (0.7, 0.3)$ on $\{0, 1\}$, and explain what the number means.
Hint: $\|\mu - \nu\|_{\text{TV}} = \frac{1}{2} \sum_x |\mu(x) - \nu(x)| = \frac{1}{2}(0.2 + 0.2) = 0.2$; it is the largest probability gap any event can detect.

A final exercise in judgment: whenever a polynomial makes a combinatorial theorem easy, ask what the polynomial made invisible. It answers questions about marginals, correlations, and counts instantly; it does not show you any single object. Decide, for one of your own problems, whether the algorithm needs a specific object or only its statistics—and if it needs the object, find the way back out.

Sources and Further Reading

Bibliography

- [1] International Mathematical Union, “Abacus Medal 2022: Mark Braverman,” official award page, 2022. <https://www.mathunion.org/imu-awards/imu-abacus-medal/abacus-medal-2022>.
- [2] International Mathematical Union, “Abacus Medal 2026: Shayan Oveis Gharan,” official award page, 2026. <https://www.mathunion.org/imu-awards/imu-abacus-medal/abacus-medal-2026>.
- [3] International Mathematical Union, “Rolf Nevanlinna Prize – Prestigious Award in Mathematical Information Sciences,” official winners list and history. <https://www.mathunion.org/imu-awards/rolf-nevanlinna-prize>.
- [4] M. Braverman and A. Rao, “Information equals amortized communication,” *Proceedings of the 55th IEEE Symposium on Foundations of Computer Science*, 2014.
- [5] M. Braverman and A. Moitra, “An information complexity approach to extended formulations,” *Proceedings of the 43rd ACM Symposium on Theory of Computing*, 2011.
- [6] M. Braverman, “Interactive information complexity,” *Foundations and Trends in Theoretical Computer Science*, 2015.
- [7] N. Christofides, “Worst-case analysis of a new heuristic for the travelling salesman problem,” technical report, Graduate School of Industrial Administration, Carnegie–Mellon University, 1976.
- [8] A. Asadi, S. Oveis Gharan, and A. Saberi, “Mixing time of the Glauber dynamics for strongly log-concave distributions,” *Proceedings of the 61st IEEE Symposium on Foundations of Computer Science*, 2020.
- [9] N. Anari, K. Liu, S. Oveis Gharan, and C. Vinzant, “Log-concave polynomials II: High-dimensional walks and an FPRAS for counting bases of a matroid,” *Proceedings of the 60th IEEE Symposium on Foundations of Computer Science*, 2019.
- [10] N. Anari, K. Liu, S. Oveis Gharan, and C. Vinzant, “Log-concave polynomials III: Mason’s conjecture for independent sets of matroids,” *Combinatorica*, 2020.
- [11] N. Anari, K. Liu, S. Oveis Gharan, and C. Vinzant, “Log-concave polynomials IV: Exchange properties, tight mixing times, and near-linear time sampling of forests,” *Proceedings of the 54th ACM Symposium on Theory of Computing*, 2022.
- [12] D. A. Spielman and S.-H. Teng, “Spectral partitioning works: Planar graphs and finite element meshes,” *Linear Algebra and its Applications*, 2007.
- [13] P. McMullen, “The maximum numbers of faces of a convex polytope,” *Mathematica*, 1970.
- [14] R. E. Tarjan, “Efficiency of a good but not linear set union algorithm,” *Journal of the ACM*, 1975.

- [15] R. E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, 1972.
- [16] L. G. Valiant, "The complexity of computing the permanent," *Theoretical Computer Science*, 1979.
- [17] L. G. Valiant, "A theory of the learnable," *Communications of the ACM*, 1984.
- [18] A. A. Razborov, "Lower bounds on the size of bounded depth networks over a complete basis with logical addition," *Mathematical Notes*, 1987.
- [19] A. A. Razborov, "Unprovability of lower bounds on the circuit size in certain fragments of bounded arithmetic," *Izvestiya: Mathematics*, 1995.
- [20] S. Goldwasser, S. Micali, and C. Rackoff, "The knowledge complexity of interactive proof systems," *SIAM Journal on Computing*, 1989.
- [21] O. Goldreich, S. Micali, and A. Wigderson, "Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems," *Journal of the ACM*, 1991.
- [22] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," *Proceedings of the 35th IEEE Symposium on Foundations of Computer Science*, 1994.
- [23] P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Physical Review A*, 1995.
- [24] M. Sudan, "Decoding of Reed Solomon codes beyond the error-correction bound," *Journal of Complexity*, 1997.
- [25] S. Arora and S. Safra, "Probabilistic checking of proofs: a new characterization of NP," *Journal of the ACM*, 1998.
- [26] J. M. Kleinberg, "Authoritative sources in a hyperlinked environment," *Journal of the ACM*, 1999.
- [27] J. M. Kleinberg, "The small-world phenomenon: an algorithmic perspective," *Proceedings of the 32nd ACM Symposium on Theory of Computing*, 2000.
- [28] J. Kleinberg, "Bursty and hierarchical structure in streams," *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2002.
- [29] D. A. Spielman and S.-H. Teng, "Smoothed analysis of algorithms: why the simplex algorithm usually takes polynomial time," *Journal of the ACM*, 2004.
- [30] D. A. Spielman and S.-H. Teng, "Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems," *Proceedings of the 36th ACM Symposium on Theory of Computing*, 2004.
- [31] D. A. Spielman, "Linear-time encodable and decodable error-correcting codes," *Proceedings of the 27th ACM Symposium on Theory of Computing*, 1996.
- [32] S. Khot, "On the power of unique 2-prover 1-round games," *Proceedings of the 34th ACM Symposium on Theory of Computing*, 2002.
- [33] S. Khot, G. Kindler, E. Mossel, and R. O'Donnell, "Optimal inapproximability results for MAX-CUT and other 2-variable CSPs?," *SIAM Journal on Computing*, 2007.
- [34] C. Daskalakis, P. W. Goldberg, and C. H. Papadimitriou, "The complexity of computing a Nash equilibrium," *SIAM Journal on Computing*, 2009.

-
- [35] C. Daskalakis, A. Deckelbaum, and C. Tzamos, “Mechanism design via optimal transport,” *Proceedings of the 14th ACM Conference on Electronic Commerce*, 2013.

Index

Braverman, Mark, 95
Daskalakis, Constantinos, 86
Khot, Subhash, 77
Kleinberg, Jon, 56
Oveis Gharan, Shayan, 102
Razborov, Alexander A., 18
Shor, Peter W., 36
Spielman, Daniel, 67
Sudan, Madhu, 46
Tarjan, Robert Endre, 1
Valiant, Leslie, 8
Wigderson, Avi, 27